

Jumpa Verse

Complete Premium SDL2 + C++ Game Documentation

Professional software engineering thesis, UML guide, beginner handbook, and technical game development documentation.

Generated from the actual project source files in `/home/error_343/Desktop/Jumpa-Verse`.

Abstract

This document explains Jumpa Verse, a 2D SDL2 and C++ side-scrolling platform game, from theory to implementation. It covers architecture, rendering, texture handling, procedural ground generation, camera illusion, trap spawning, menu screens, login and registration, score storage, fullscreen behavior, clamping, random generation, and the complete main loop. The report is written for beginners while preserving professional software architecture detail.

Acknowledgement

This report acknowledges the SDL2 ecosystem, the C++ standard library, and the project implementation effort represented by the Jumba Verse source files. The explanations are designed to help a beginner understand the codebase and also help an evaluator inspect the software engineering decisions.

Table of Contents

1. Chapter 1 - INTRODUCTION TO THE GAME
2. Chapter 2 - COLOR SCHEME, UI STYLE, FONT SYSTEM
3. Chapter 3 - TEXTURE SYSTEM AND GROUND TEXTURING
4. Chapter 4 - GROUND GENERATION SYSTEM
5. Chapter 5 - CAMERA SYSTEM
6. Chapter 6 - TRAP SYSTEM
7. Chapter 7 - MENU SYSTEM
8. Chapter 8 - PLAYER SYSTEM AND MOVEMENT
9. Chapter 9 - TEXT RENDERING ISSUES
10. Chapter 10 - ERROR HANDLING SYSTEM
11. Chapter 11 - LOGIN SCREEN AND GAME OVER SCREEN
12. Chapter 12 - FILE HANDLING
13. Chapter 13 - LOGIN/REGISTRATION ANIMATIONS
14. Chapter 14 - COMPLETE GAME FLOW
15. Chapter 15 - RANDOM NUMBER GENERATION
16. Chapter 16 - LIBRARIES USED
17. Chapter 17 - FULLSCREEN IMPLEMENTATION
18. Chapter 18 - CLAMPING SYSTEM
19. Chapter 19 - MAIN.CPP COMPLETE FLOW

List of Figures

1. Figure 1: Game Architecture Overview
2. Figure 2: Gameplay Flowchart
3. Figure 3: Real-Time Game Loop
4. Figure 4: Character State Diagram
5. Figure 5: Color Palette and Contrast
6. Figure 6: Main Menu UI Mockup
7. Figure 7: Typography Role Comparison
8. Figure 8: Ground Texture Tiling Atlas
9. Figure 9: Texture Pointer Memory Model
10. Figure 10: Texture Rendering Pipeline
11. Figure 11: Texture and Ground UML
12. Figure 12: Ground Lifecycle
13. Figure 13: Vector Storage and Recycling
14. Figure 14: Ground Generation Sequence
15. Figure 15: Camera Illusion
16. Figure 16: Before and After Offset
17. Figure 17: Coordinate Conversion
18. Figure 18: Trap Collision and Hitbox
19. Figure 19: Trap Probability and Attempts
20. Figure 20: Trap Lifecycle
21. Figure 21: Trap UML and Relationships
22. Figure 22: Menu Screen Composition
23. Figure 23: Button Hover State
24. Figure 24: Menu State Transitions
25. Figure 25: Player Movement Forces
26. Figure 26: Physics Flowchart
27. Figure 27: Player Boolean State
28. Figure 28: Text Rendering Pipeline
29. Figure 29: Font Fallback System
30. Figure 30: Initialization Flowchart

- 31. Figure 31: Error Handling Branches
- 32. Figure 32: Screen Transitions
- 33. Figure 33: Namespace Helper Boundary
- 34. Figure 34: File IO Data Flow
- 35. Figure 35: Read Write System
- 36. Figure 36: UI Animation Timeline
- 37. Figure 37: Frame Progression
- 38. Figure 38: Master Architecture
- 39. Figure 39: Complete Program Sequence
- 40. Figure 40: RNG Flowchart
- 41. Figure 41: Probability Systems
- 42. Figure 42: Library Dependency Diagram
- 43. Figure 43: Header Role Map
- 44. Figure 44: Fullscreen Toggle Flow
- 45. Figure 45: Resolution Comparison
- 46. Figure 46: Coordinate Limits
- 47. Figure 47: Boundary Control Flow
- 48. Figure 48: Main.cpp Execution Flow
- 49. Figure 49: Physics and Rendering Cycle

List of Tables

1. SDL2 and extension functions used by Jumpa Verse
2. Library roles and project responsibilities

Chapter 1 - INTRODUCTION TO THE GAME

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is main.cpp, object.h. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

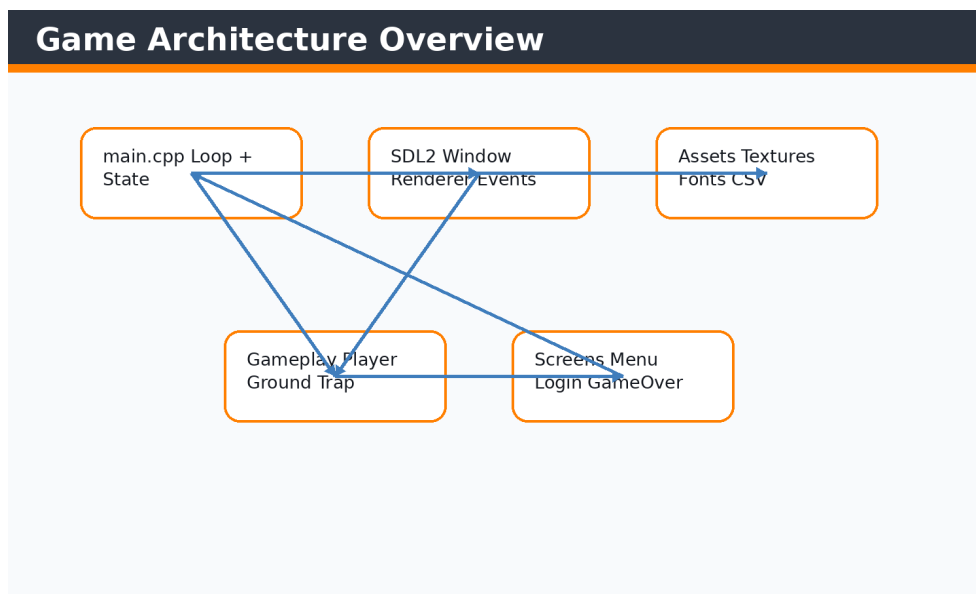


Figure 1.1 Game

architecture diagram

The architecture places main.cpp as the coordinator between SDL2, gameplay entities, screen states, and asset files.

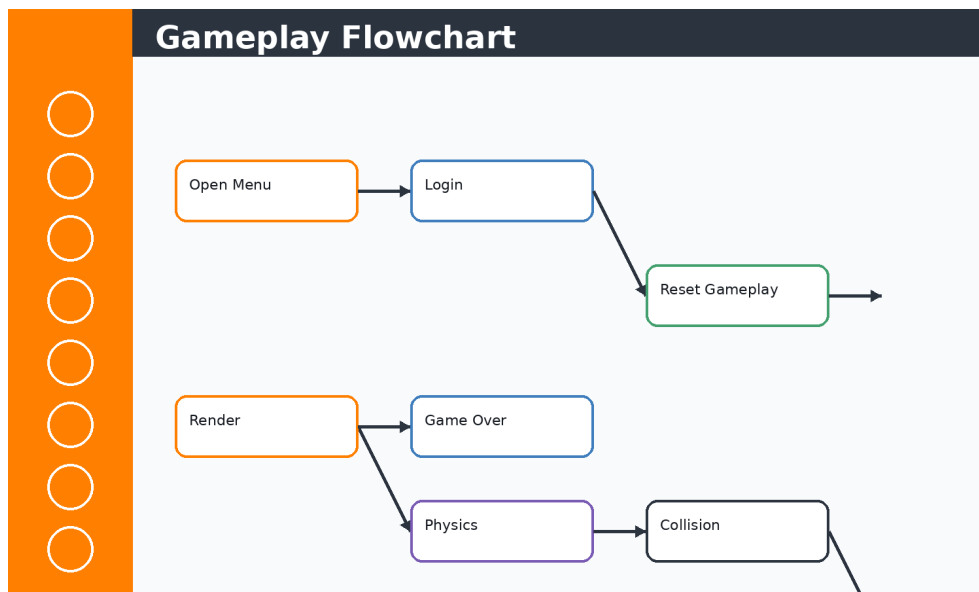


Figure 1.2 Gameplay

flowchart

This flow matches the nested menu loop and active gameplay loop in main.cpp.

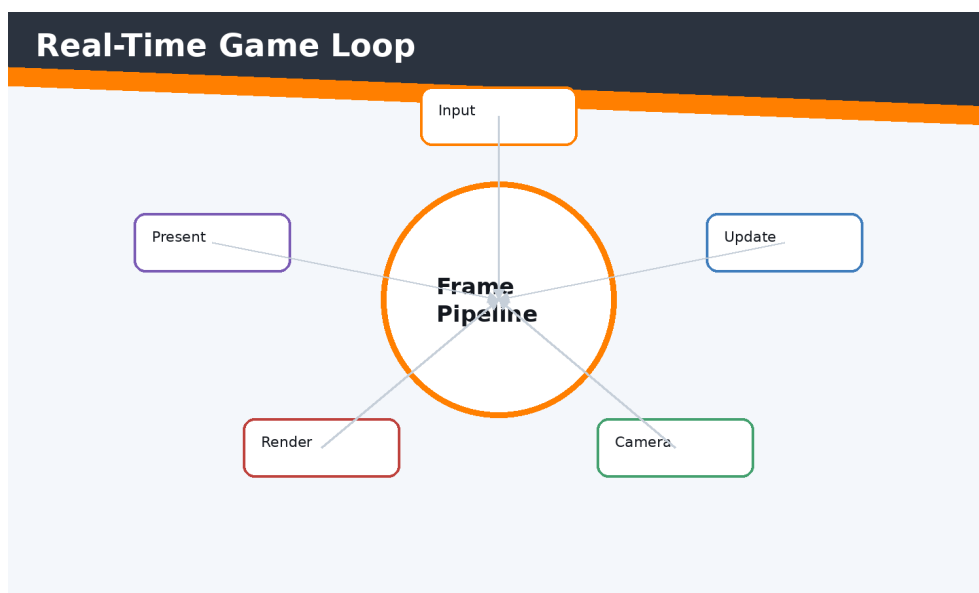


Figure 1.3 Main game

loop diagram

Each frame moves from event input to update logic to rendering and presentation.

Character State Diagram

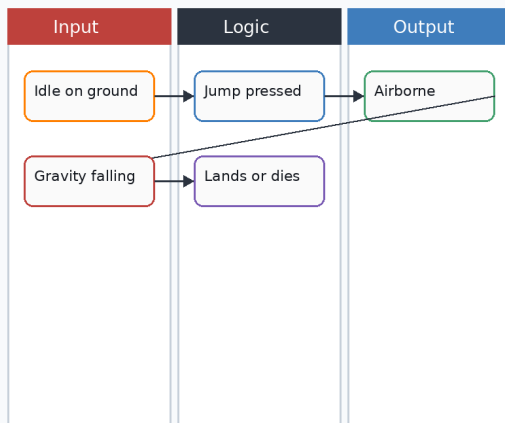


Figure 1.4 Character

state diagram

The player state is expressed by `onGround`, `jumpVelocity`, `gravity`, and collision checks.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAII-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

Jumpa Verse is a side-scrolling survival platform game. The player is a rectangle-like avatar that jumps over procedurally placed spike traps while the world scrolls left. Death occurs when the player falls below the visible play field or intersects a trap hitbox.

Code Excerpt: `main.cpp`

```
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include <iostream>
#include <SDL2/SDL.h>
#include <iterator>
#include <vector>
#include <string.h>
#include <vector>
```

```

#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include "functions.h"
#include<stdlib.h>
#include<SDL2/SDL_ttf.h>
float cammeraDash=0;
bool exitb = false;
int main()
{
    SDL_Window* window=nullptr;
    SDL_Renderer* renderer=nullptr;

    player player1;

    if(!init(window,renderer))
    {
        std::cerr<<"FAILED TO START "<<std::endl;
        return -1;
    }

    float cammera_offSet=0.5f; //no 2d camera in sdl2 so we create an illusion
    // ground_Class
    ground(100,900,500,100,renderer,"Images_textures/ground.jpeg","Images_textures/
underGround.jpeg");

    // ground=groundRandomgenerator(ground,renderer);
    SDL_Texture* groundTex = IMG_LoadTexture(renderer, "Images_textures/ground.jpeg");
    SDL_Texture* ugTex = IMG_LoadTexture(renderer, "Images_textures/underGround.jpeg");
    TTF_Font *font =TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf",100);
    std::vector<trapSpike> trap;
    std::vector<ground_Class> ground;
    Score score;
    GameOver gameOver;
    auto resetGameplay = [&]()
    {
        player1 = player();
        cammera_offSet = 0.5f;
        trap.clear();
        ground = ground_generator(renderer, groundTex, ugTex);
        groundRandomgenerator(renderer, groundTex, ugTex, ground, trap);
        // ground.push_back(temp);
        resetScore(score);
    }
}

```

```

        gameOver.reset();
    };

    initializeScore(score);
    resetGameplay();
    bool quit=false;
    SDL_Event eventManager;
    Uint32 lastTime = SDL_GetTicks();
    int i=0;
    int count=0;
    const float targetFrameTime = 1000.0f / 60.0f; // 16.67 ms
    bool menu=false;
    while(!quit)
    {

        Uint32 frameStart = SDL_GetTicks();
        int screenW = currentScreenWidth;
        int screenH = currentScreenHeight;
        getScreenSize(renderer, screenW, screenH);
        // ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: object.h

```

#pragma once

#include <SDL2/SDL.h>
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_image.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_surface.h>
#include <iostream>
#include <vector>
#include <string>
#include <SDL2/SDL2_gfxPrimitives.h> //to import trigons
#include <SDL2/SDL_ttf.h>

extern int currentScreenWidth;
extern int currentScreenHeight;

```

```
inline int scaledGroundTileSize()
{
    if (currentScreenWidth >= 3840)
    {
        return 96;
    }
    if (currentScreenWidth >= 3200)
    {
        return 80;
    }
    if (currentScreenWidth >= 2400)
    {
        return 64;
    }
    return 50;
}
```

```
inline int scaledTrapSize()
{
    if (currentScreenWidth >= 3200)
    {
        return 88;
    }
    if (currentScreenWidth >= 2400)
    {
        return 68;
    }
    return 50;
}
```

```
inline int scaledPlayerSize()
{
    if (currentScreenWidth >= 3200)
    {
        return 88;
    }
    if (currentScreenWidth >= 2400)
    {
        return 68;
    }
    return 50;
}
```

```
class hitBox
{
```

```
public:
    int x, y, w, h;

    hitBox(int x, int y, int w, int h)
    {
        this->x = x;
        this->y = y;
        this->w = w;
        this->h = h;
    }
    // ... excerpt trimmed for the report ...
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 2 - COLOR SCHEME, UI STYLE, FONT SYSTEM

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is menuScreen.cpp, score.cpp. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

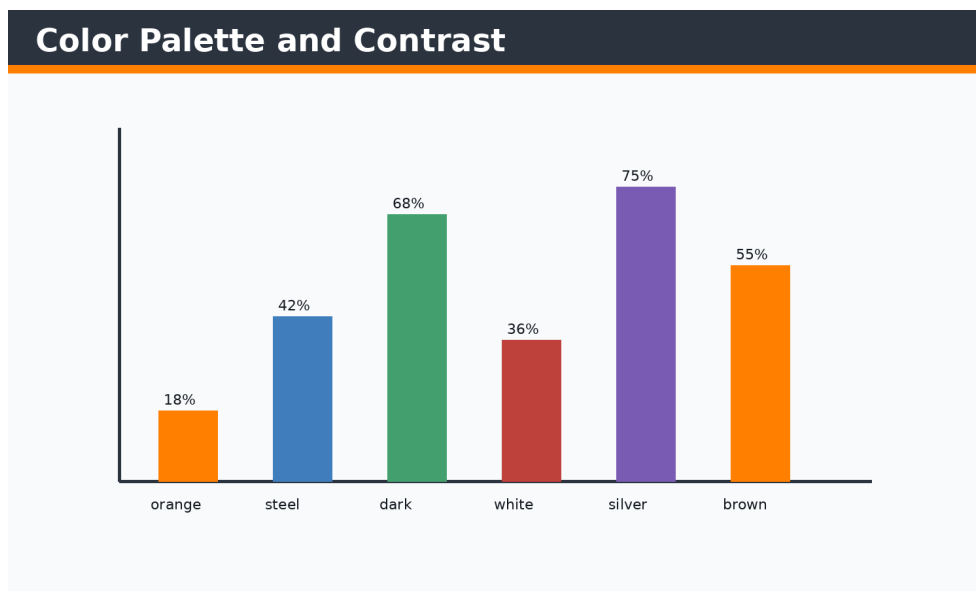


Figure 2.1 Color palette

figure

Orange is the action color; dark gray is the stable background; white and silver maintain readability.

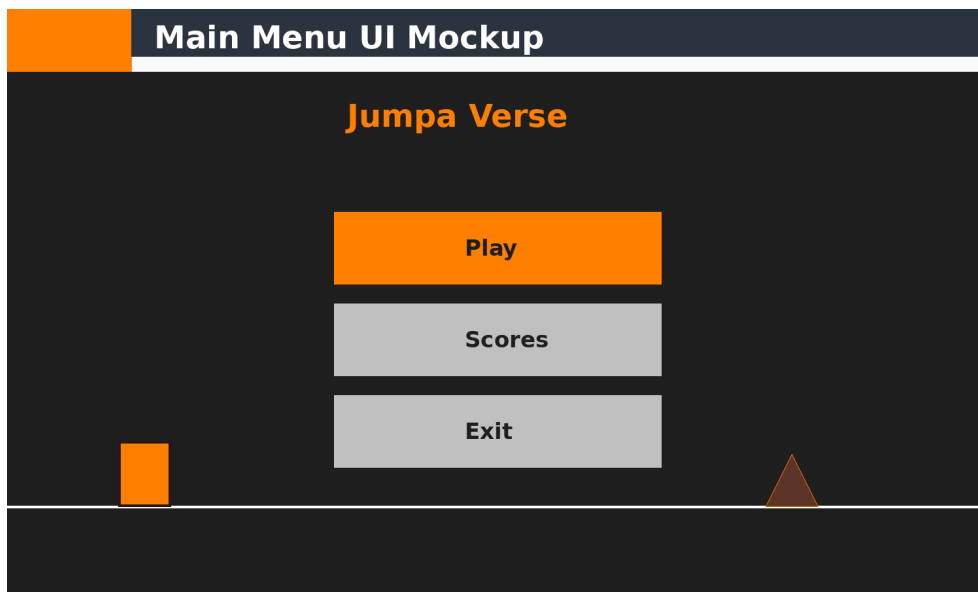


Figure 2.2 UI preview

layout

The mockup mirrors menuScreen.cpp with title, Play/Scores/Exit buttons, ground line, cube, and animated trap preview.

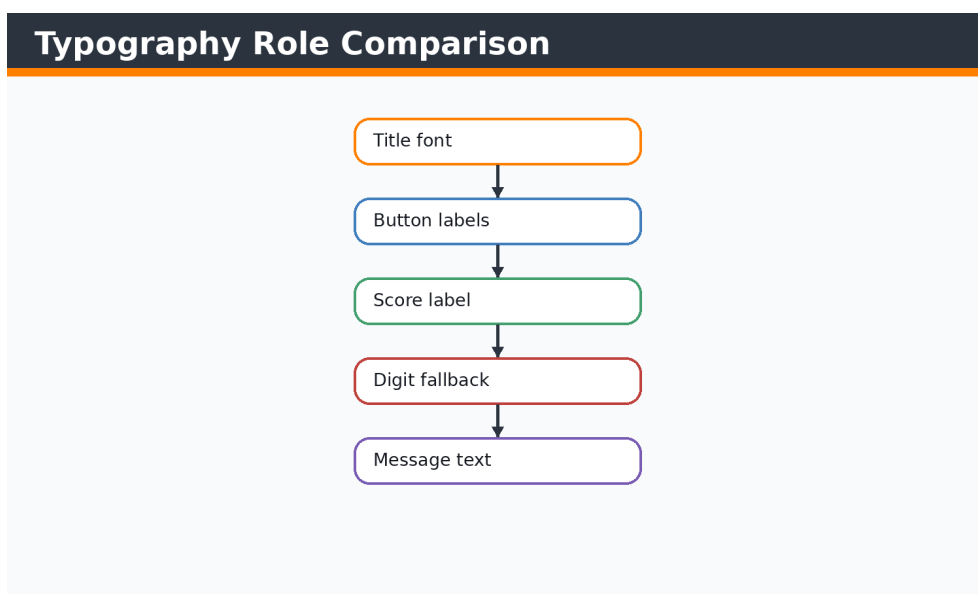


Figure 2.3 Typography

comparison

Different text roles are handled by TTF_OpenFont and text-to-texture rendering.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAII-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

The UI uses a high-contrast dark background, orange highlight, silver inactive buttons, and white text. The custom Debrosee typeface gives title identity, while system fonts are used as numeric fallbacks so integer scores remain readable.

Code Excerpt: menuScreen.cpp

```
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>
#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include<SDL2/SDL2_gfxPrimitives.h>
#include<random>
#include "functions.h"

SDL_Color mainMenuColor::background = {30, 30, 30, 255};
SDL_Color mainMenuColor::buttonNormal = {70, 130, 180, 255};
SDL_Color mainMenuColor::buttonHover = {100, 160, 210, 255};
SDL_Color mainMenuColor::buttonActive = {40, 90, 140, 255};
SDL_Color mainMenuColor::text = {255, 255, 255, 255};

namespace
{
    bool openLoginForPlay(SDL_Renderer* renderer, TTF_Font* font)
    {
        LoginScreenAction action = runLoginScreen(renderer, font);
        if (action == LoginScreenAction::Quit)
        {
            exitb = true;
            return true;
        }

        return action == LoginScreenAction::LoginSuccess;
    }
}

//Run
```

```

bool menuScreen(SDL_Renderer* renderer, TTF_Font* font)
{
    optionBoxes playButton(740, 430, 500, 150);
    playButton.box_Initializer(renderer, font, "Play");
    SDL_Texture* titleTexture = nullptr;
    if (font)
    {
        SDL_Surface* titleSurface = TTF_RenderText_Blended(font, "Jumpa Verse", SDL_Color{255,
127, 0, 255});
        if (titleSurface)
        {
            titleTexture = SDL_CreateTextureFromSurface(renderer, titleSurface);
            SDL_FreeSurface(titleSurface);
        }
    }
    int screenW = currentScreenWidth;
    int screenH = currentScreenHeight;
    getScreenSize(renderer, screenW, screenH);
    SDL_Point start={0, screenH - 160};
    SDL_Point end={screenW, screenH - 160};

    bool running = true;
    SDL_Event event;
    int mousex,mousey;
    int selectedButton = 0;
    bool playSelected = true;
    bool scoresSelected = false;
    bool exitSelected = false;
    Uint32 lastFrameTime = SDL_GetTicks();
    float cubeVelocity = 0.0f;
    float cubeY = (float)(start.y - 45);
    float trapOffset = 0.0f;
    int lastSimulationLineY = start.y;

    // ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: score.cpp

```

#include "object.h"

```

```
#include "functions.h"

#include <SDL2/SDL_ttf.h>
#include <SDL2/SDL.h>
#include <string>

Score::Score()
{
    initialize();
}

Score::~Score() = default;

void Score::initialize()
{
    reset();
}

void Score::update(float deltaTime, bool playerAlive)
{
    if (!playerAlive || deltaTime <= 0.0f)
    {
        return;
    }

    timeAccumulator += deltaTime;
    if (timeAccumulator >= 1.0f)
    {
        score += 1;
        timeAccumulator = 0.0f;
    }
}

void Score::render(SDL_Renderer* renderer, TTF_Font* font)
{
    if (!renderer)
    {
        return;
    }

    static TTF_Font* labelFont = nullptr;
    if (!labelFont)
    {
        labelFont = TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf", 26);
        if (!labelFont)
```

```

    {
        labelFont = font;
    }
}

static TTF_Font* digitFont = nullptr;
if (!digitFont)
{
    digitFont = TTF_OpenFont("/usr/share/fonts/liberation/LiberationSans-Bold.ttf", 26);
    if (!digitFont)
    {
        digitFont = TTF_OpenFont("/usr/share/fonts/TTF/DejaVuSans-Bold.ttf", 26);
    }
}

if (!labelFont || !digitFont)
{
    return;
}

int screenW = currentScreenWidth;
int screenH = currentScreenHeight;
getScreenSize(renderer, screenW, screenH);

// ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 3 - TEXTURE SYSTEM AND GROUND TEXTURING

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumba Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is object.h, main.cpp. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

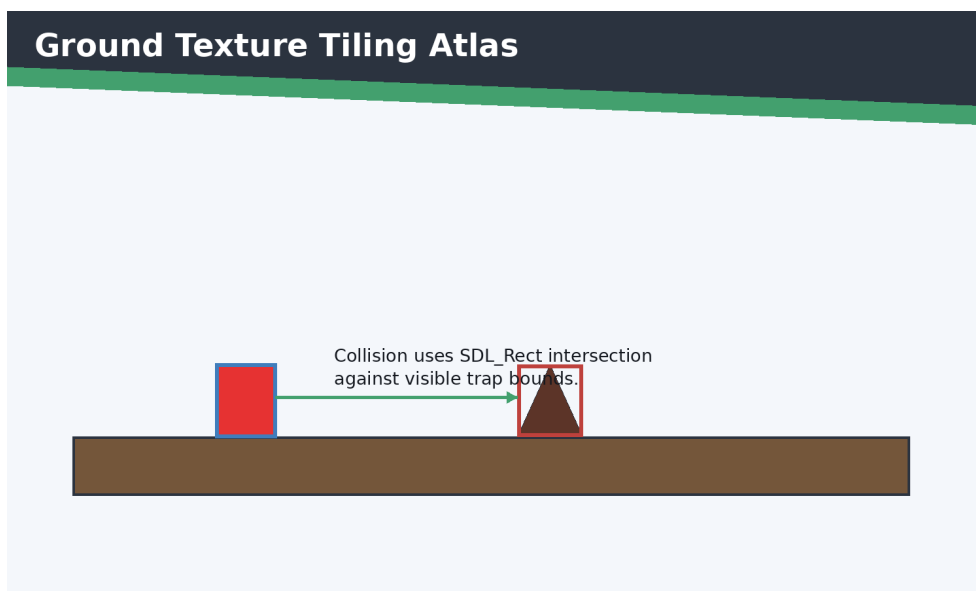


Figure 3.1 Texture

atlas and tiling visual

The platform surface is repeated with SDL_RenderCopy into individual destination rectangles.

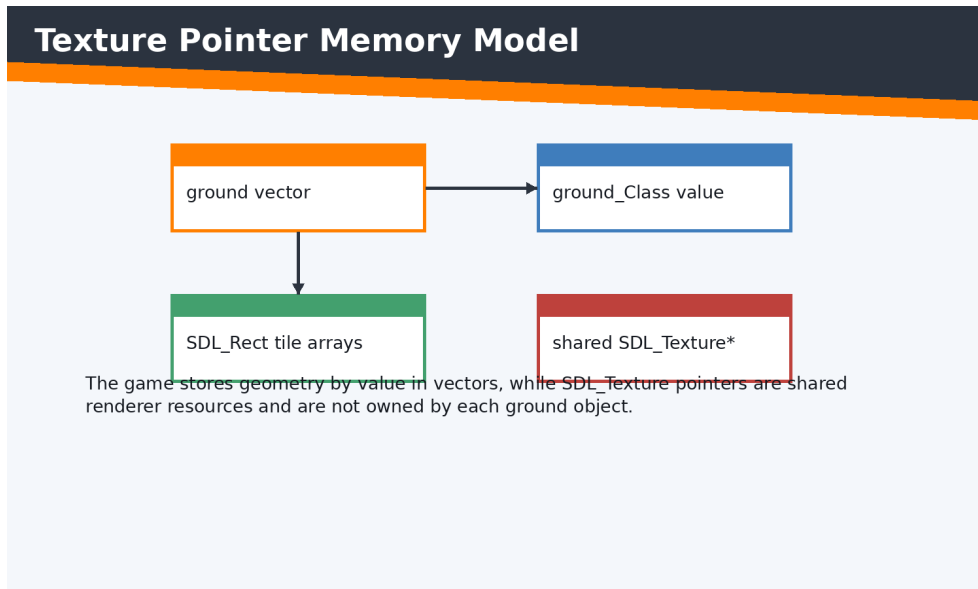


Figure 3.2 Texture

memory layout

The diagram separates object geometry from shared renderer-owned texture resources.

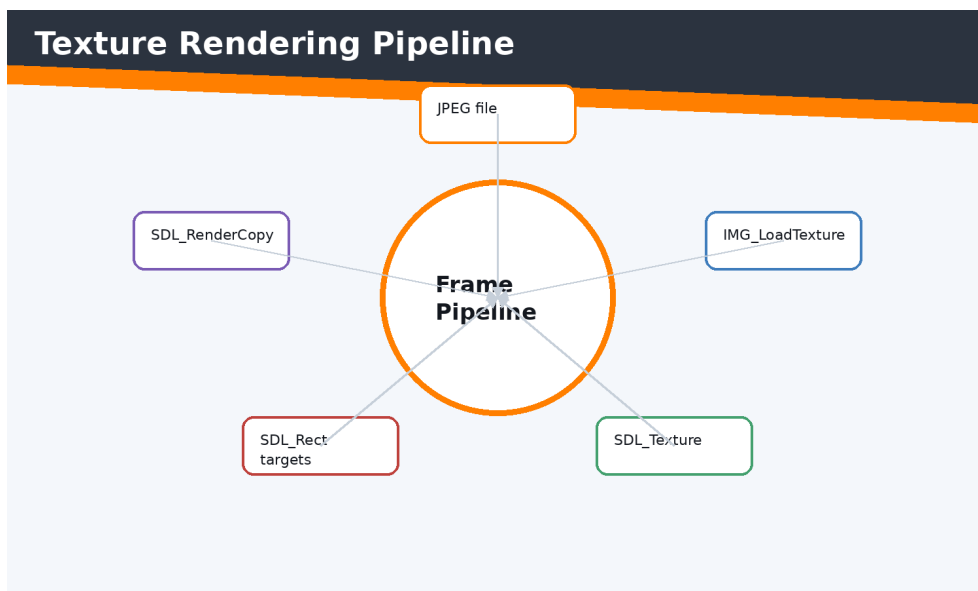


Figure 3.3 Rendering

flow diagram

Texture pixels move from image files into GPU textures, then are copied to rectangles every frame.

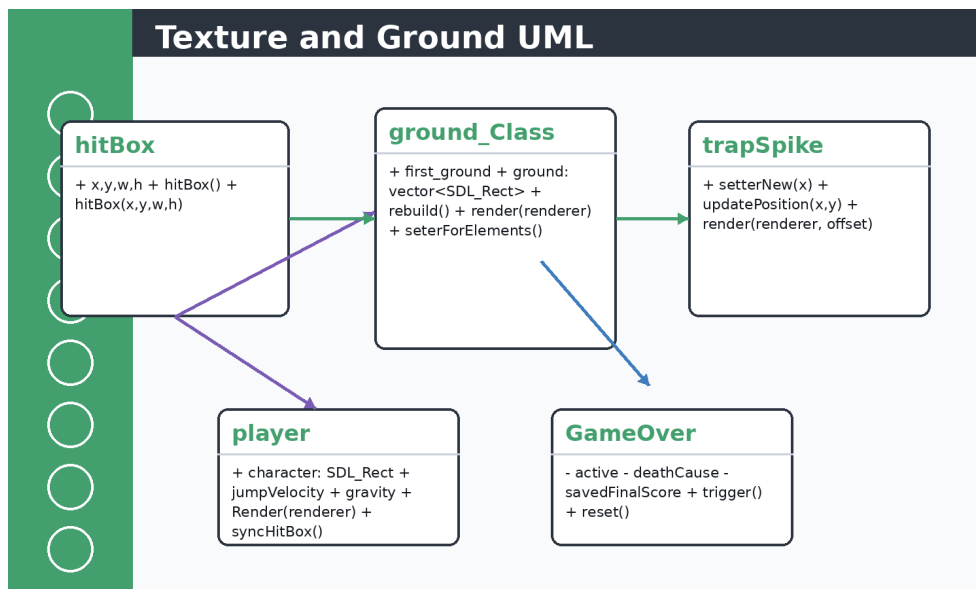


Figure 3.4 UML

diagram for texture-ground relation

ground_Class composes rectangle vectors and associates with shared SDL_Texture pointers.

SDL2 and extension functions used by Jumpa Verse

<code>SDL_Init(Uint32 flags)</code>	int	Initializes SDL subsystems; called in init() before creating the window.
<code>SDL_CreateWindow(title,x,y,w,h,flags)</code>	SDL_Window*	Allocates the native game window and applies fullscreen/resizable flags.
<code>SDL_CreateRenderer(window,index,flags)</code>	SDL_Renderer*	Creates the accelerated rendering context used by all draw calls.
<code>SDL_PollEvent(SDL_Event*)</code>	int	Pulls queued keyboard, mouse, text, quit, and window events inside each loop.
<code>SDL_GetKeyboardState(int*)</code>	const Uint8*	Returns the current keyboard state array used by physicsDevelop().
<code>SDL_GetMouseState(int*,int*)</code>	Uint32	Reads mouse position/buttons for menu hover and player jump shortcuts.

<code>SDL_SetRenderDrawColor(renderer, r, g, b, a)</code>	int	Selects the drawing color for clear, rectangles, borders, and UI.
<code>SDL_RenderClear(renderer)</code>	int	Clears the current back buffer before drawing the next frame.
<code>SDL_RenderFillRect(renderer, rect)</code>	int	Draws filled UI buttons, player body, and panels.
<code>SDL_RenderDrawRect(renderer, rect)</code>	int	Draws borders for score boxes, controls, and hitbox-style UI.
<code>SDL_RenderCopy(renderer, texture, src, dst)</code>	int	Copies texture pixels into a target rectangle for ground and text.
<code>SDL_RenderPresent(renderer)</code>	void	Swaps the rendered back buffer to the visible screen.
<code>IMG_Init(flags)</code>	int	Initializes SDL_image JPEG support for ground texture loading.
<code>IMG_LoadTexture(renderer, path)</code>	SDL_Texture*	Loads image files directly into GPU texture resources.
<code>TTF_OpenFont(path, size)</code>	TTF_Font*	Loads typefaces used by menus, score display, and screens.
<code>TTF_RenderText_Blended(font, text, color)</code>	SDL_Surface*	Rasterizes text into a temporary CPU surface.
<code>SDL_CreateTextureFromSurface(renderer, surface)</code>	SDL_Texture*	Uploads the text surface into a GPU texture.
<code>filledTrigonRGBA(renderer, ...)</code>	int	SDL2_gfx filled triangle used for solid spike traps.
<code>trigonRGBA(renderer, ...)</code>	int	SDL2_gfx outline triangle used for trap edges.
<code>SDL_SetWindowFullscreen(window, ...)</code>	int	Toggles desktop fullscreen and

flags)

windowed mode in custom controls.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAI-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

Ground rendering divides each platform into `SDL_Rect` tiles. The top row uses `ground.jpeg` and the lower rows use `underGround.jpeg`. Textures are loaded once in `main.cpp` and shared by pointer to avoid expensive duplicated GPU allocations.

Code Excerpt: `object.h`

```
#pragma once

#include <SDL2/SDL.h>
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_image.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_surface.h>
#include <iostream>
#include <vector>
#include <string>
#include <SDL2/SDL2_gfxPrimitives.h> //to import trigons
#include <SDL2/SDL_ttf.h>

extern int currentScreenWidth;
extern int currentScreenHeight;

inline int scaledGroundTileSize()
{
    if (currentScreenWidth >= 3840)
    {
        return 96;
    }
}
```

```
    if (currentScreenWidth >= 3200)
    {
        return 80;
    }
    if (currentScreenWidth >= 2400)
    {
        return 64;
    }
    return 50;
}

inline int scaledTrapSize()
{
    if (currentScreenWidth >= 3200)
    {
        return 88;
    }
    if (currentScreenWidth >= 2400)
    {
        return 68;
    }
    return 50;
}

inline int scaledPlayerSize()
{
    if (currentScreenWidth >= 3200)
    {
        return 88;
    }
    if (currentScreenWidth >= 2400)
    {
        return 68;
    }
    return 50;
}

class hitBox
{
public:
    int x, y, w, h;

    hitBox(int x, int y, int w, int h)
    {
        this->x = x;
```

```
        this->y = y;
        this->w = w;
        this->h = h;
// ... excerpt trimmed for the report ...
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: main.cpp

```
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>
#include<vector>
#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include "functions.h"
#include<stdlib.h>
#include<SDL2/SDL_ttf.h>
float cammeraDash=0;
bool exitb = false;
int main()
{
    SDL_Window* window=nullptr;
    SDL_Renderer* renderer=nullptr;

    player player1;

    if(!init(window,renderer))
    {
        std::cerr<<"FAILED TO START "<<std::endl;
        return -1;
    }

    float cammera_offSet=0.5f; //no 2d camera in sdl2 so we create an illusion
    // ground_Class
```

```

ground(100,900,500,100,renderer,"Images_textures/ground.jpeg","Images_textures/
underGround.jpeg");

//    ground=groundRandomgenerator(ground,renderer);
SDL_Texture* groundTex = IMG_LoadTexture(renderer, "Images_textures/ground.jpeg");
SDL_Texture* ugTex = IMG_LoadTexture(renderer, "Images_textures/underGround.jpeg");
TTF_Font *font =TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf",100);
std::vector<trapSpike> trap;
std::vector<ground_Class> ground;
Score score;
GameOver gameOver;
auto resetGameplay = [&]()
{
    player1 = player();
    cammera_offSet = 0.5f;
    trap.clear();
    ground = ground_generator(renderer, groundTex, ugTex);
    groundRandomgenerator(renderer, groundTex, ugTex, ground, trap);
    // ground.push_back(temp);
    resetScore(score);
    gameOver.reset();
};

initializeScore(score);
resetGameplay();
bool quit=false;
SDL_Event eventManager;
Uint32 lastTime = SDL_GetTicks();
int i=0;
int count=0;
const float targetFrameTime = 1000.0f / 60.0f; // 16.67 ms
bool menu=false;
while(!quit)
{

    Uint32 frameStart = SDL_GetTicks();
    int screenW = currentScreenWidth;
    int screenH = currentScreenHeight;
    getScreenSize(renderer, screenW, screenH);
    // ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 4 - GROUND GENERATION SYSTEM

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is ground.cpp, object.h. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

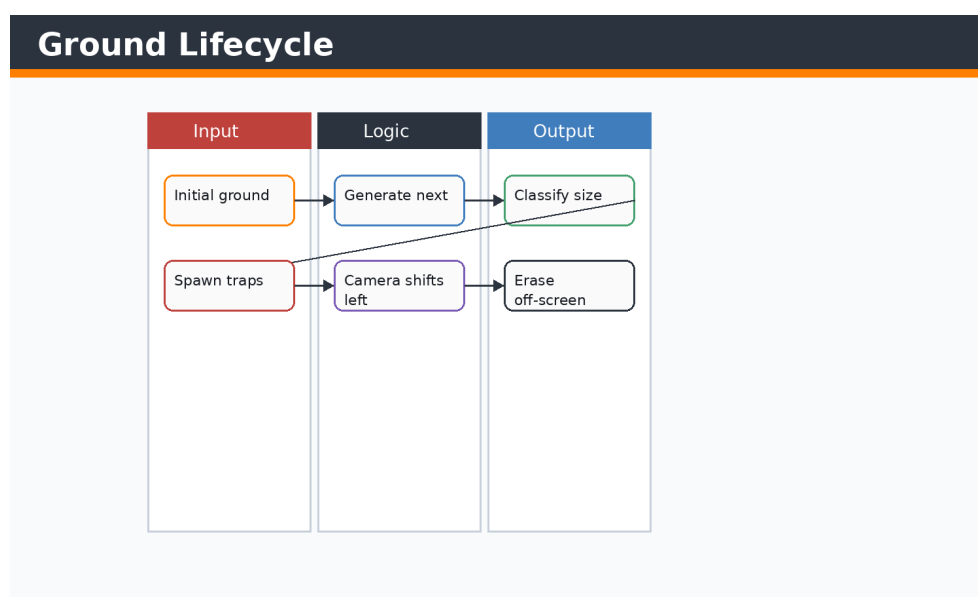


Figure 4.1 Ground

lifecycle diagram

The lifecycle directly follows ground_generator, groundRandomgenerator, cameraMovmentObj, and freeMemory.

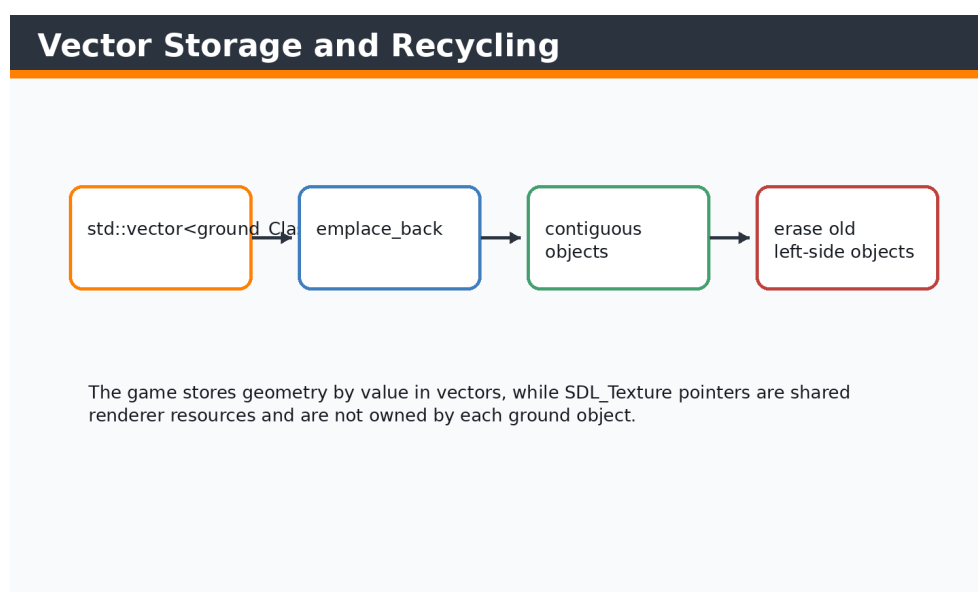


Figure 4.2 Vector

memory diagram

The vector stores platform objects by value and shrinks as old platforms are removed.

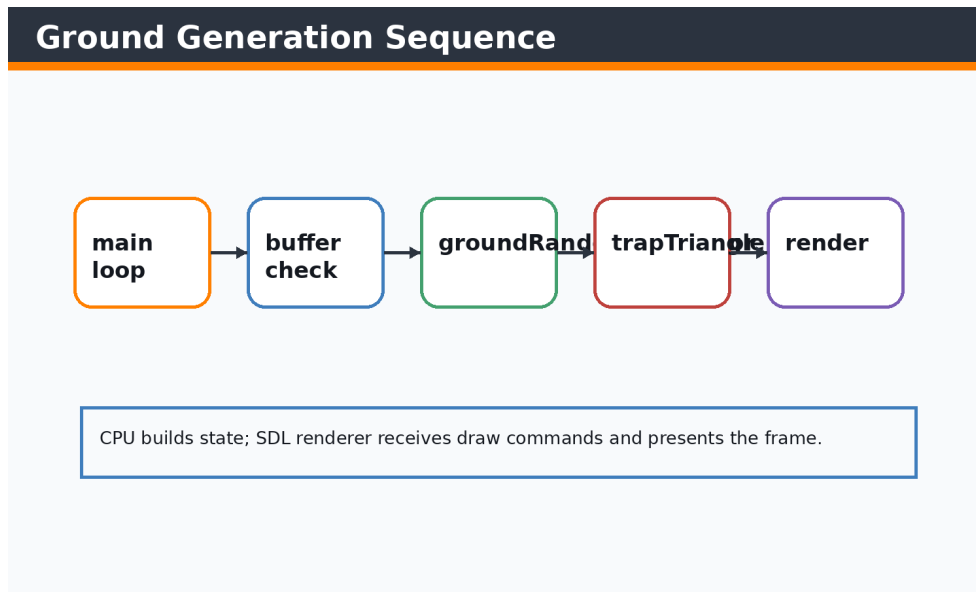


Figure 4.3 Sequence

diagram

main.cpp generates new platforms when the last platform does not extend far enough ahead.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAII-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

Ground generation uses `std::random_device`, `std::mt19937`, and uniform distributions to create platform width, height, gap, and y-position. Existing platforms are recycled by erasing objects that leave the left side of the screen.

Code Excerpt: ground.cpp

```
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
```

```
#include <iterator>
#include<vector>
#include<string.h>

#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include<random>
#include "functions.h"

namespace
{
    int clampInt(int value, int minValue, int maxValue)
    {
        if (value < minValue)
        {
            return minValue;
        }
        if (value > maxValue)
        {
            return maxValue;
        }
        return value;
    }
}

std::vector<ground_Class> ground_generator(SDL_Renderer* renderer, SDL_Texture*
groundTex, SDL_Texture* ugTex)
{
    int screenW = currentScreenWidth;
    int screenH = currentScreenHeight;
    getScreenSize(renderer, screenW, screenH);

    std::random_device rd;
    std::mt19937 gen(rd());

    const int minY = clampInt((int)(screenH * 0.40f), 160, screenH - 260);
    const int maxY = clampInt((int)(screenH * 0.72f), minY + 50, screenH - 120);
    const int minW = clampInt((int)(screenW * 0.52f), 700, 1700);
    const int maxW = clampInt((int)(screenW * 0.95f), minW, 3000);
    const int maxH = clampInt((int)(screenH * 0.30f), 140, 460);

    std::uniform_int_distribution<> disty(minY, maxY);
    std::uniform_int_distribution<> distw(minW, maxW);
    std::uniform_int_distribution<> disth(50, maxH);
```



```

    int x = 0;
    int y = disty(gen);
    int w = distw(gen);
    int h = disth(gen);

    std::vector<ground_Class> ground;

    ground.emplace_back(
        x, y, w, h,
        groundTex,ugTex
    );
    ground[0].first_ground=true;

    return ground;
}

void groundRandomgenerator(SDL_Renderer* renderer,SDL_Texture* groundTex,SDL_Texture*
ugTex,std::vector<ground_Class> &prevGround,std::vector<trapSpike>& trap)
{
    if (prevGround.empty())
// ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: object.h

```

#pragma once

#include <SDL2/SDL.h>
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_image.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_surface.h>
#include <iostream>
#include <vector>
#include <string>
#include <SDL2/SDL2_gfxPrimitives.h> //to import trigons
#include<SDL2/SDL_ttf.h>

```

```
extern int currentScreenWidth;  
extern int currentScreenHeight;  
  
inline int scaledGroundTileSize()  
{  
    if (currentScreenWidth >= 3840)  
    {  
        return 96;  
    }  
    if (currentScreenWidth >= 3200)  
    {  
        return 80;  
    }  
    if (currentScreenWidth >= 2400)  
    {  
        return 64;  
    }  
    return 50;  
}  
  
inline int scaledTrapSize()  
{  
    if (currentScreenWidth >= 3200)  
    {  
        return 88;  
    }  
    if (currentScreenWidth >= 2400)  
    {  
        return 68;  
    }  
    return 50;  
}  
  
inline int scaledPlayerSize()  
{  
    if (currentScreenWidth >= 3200)  
    {  
        return 88;  
    }  
    if (currentScreenWidth >= 2400)  
    {  
        return 68;  
    }  
    return 50;  
}
```

```
class hitBox
{
public:
    int x, y, w, h;

    hitBox(int x, int y, int w, int h)
    {
        this->x = x;
        this->y = y;
        this->w = w;
        this->h = h;
    }
// ... excerpt trimmed for the report ...
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 5 - CAMERA SYSTEM

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is camera.cpp, main.cpp. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

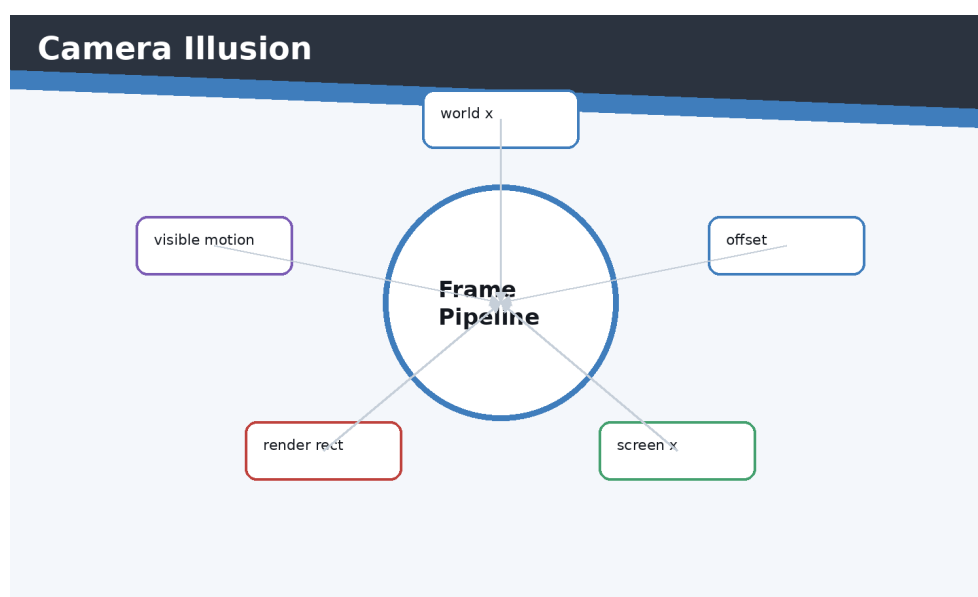


Figure 5.1 Camera

illusion diagram

The side-scrolling effect is created by changing object coordinates before rendering.

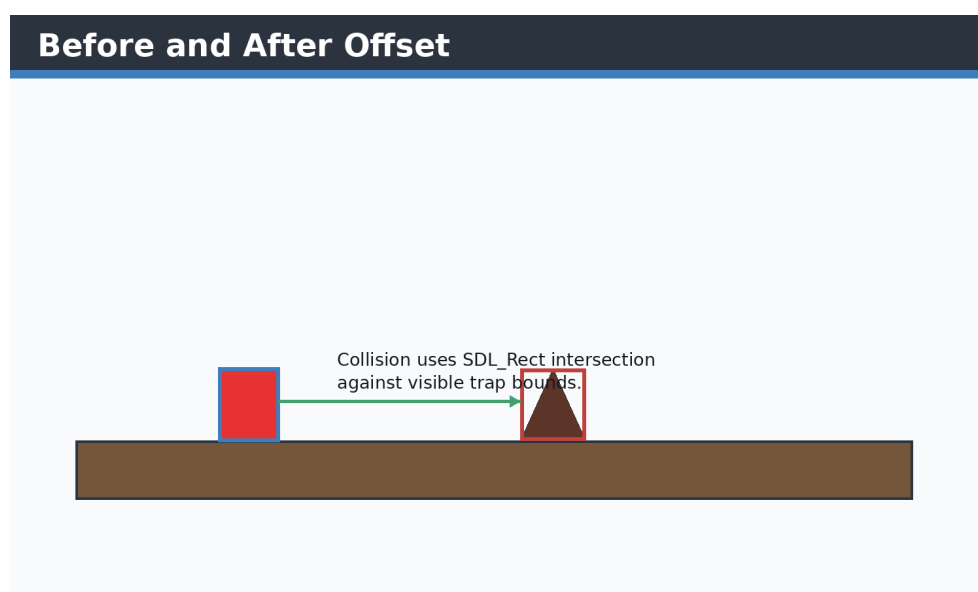


Figure 5.2 Side

scrolling figure

Ground and traps shift together, preserving relative positions.

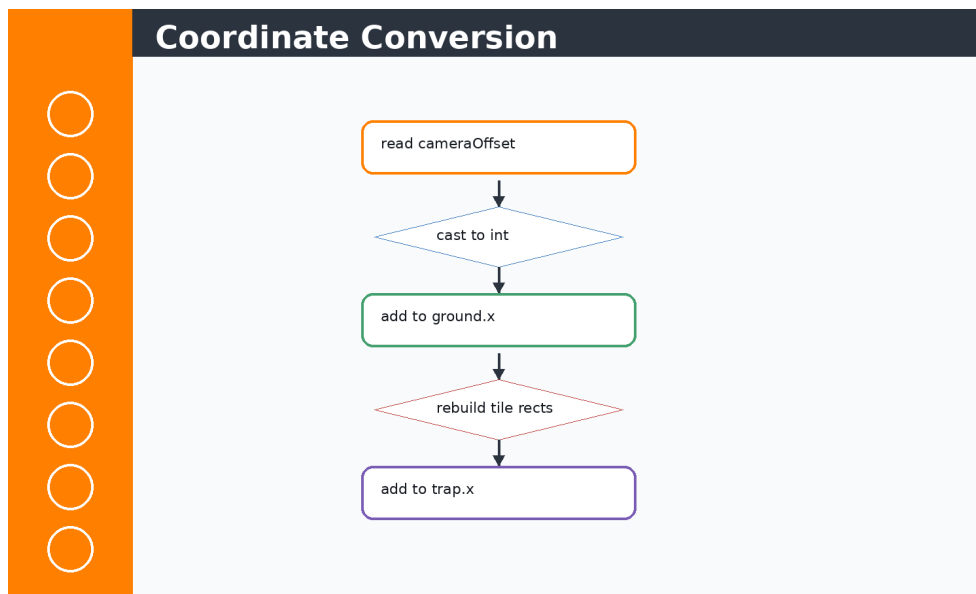


Figure 5.3 Coordinate

conversion visual

cameraMovmentObj updates platform geometry and trap x positions in the same pass.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAI-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

SDL2 does not provide a game camera. Jumpa Verse simulates one by shifting world objects left using camera offsets while the player remains visually near the same screen region.

Code Excerpt: camera.cpp

```
#include<iostream>
#include<vector>
#include<SDL2/SDL.h>
#include "object.h"
#include "functions.h"
```

```

void cameraMovmentObj(std::vector<ground_Class>& ground, float cameraOffset,
std::vector<trapSpike>& trap)
{
    int intOffset = (int)cameraOffset;
    if (intOffset == 0 && cameraOffset != 0)
    {
        // If the offset is fractional but less than 1, we might want to still move
        // but since x is int, we can't. However, we can at least ensure ground and traps move
        together.
    }

    for (size_t i = 0; i < ground.size(); i++)
    {
        ground[i].setterForElements(
            ground[i].x + intOffset,
            ground[i].y,
            ground[i].w,
            ground[i].h
        );
    }

    for (size_t i = 0; i < trap.size(); i++)
    {
        trap[i].setterNew(trap[i].x + intOffset);
    }
}

// std::vector<ground_Class>(std::vector<ground_Class> ground, float cameraOffset)
// {
//     for (int i = 0; i < ground.size(); i++)
//     {
//         ground[i].setterForElements(
//             ground[i].x + (int)cameraOffset,
//             ground[i].y,
//             ground[i].w,
//             ground[i].h
//         );
//     }
// }
//return ground;
// }

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction:

main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: main.cpp

```
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>
#include<vector>
#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include "functions.h"
#include<stdlib.h>
#include<SDL2/SDL_ttf.h>
float cammeraDash=0;
bool exitb = false;
int main()
{
    SDL_Window* window=NULLptr;
    SDL_Renderer* renderer=NULLptr;

    player player1;

    if(!init(window,renderer))
    {
        std::cerr<<"FAILED TO START "<<std::endl;
        return -1;
    }

    float cammera_offSet=0.5f; //no 2d camera in sdl2 so we create an illusion
    // ground_Class
    ground(100,900,500,100,renderer,"Images_textures/ground.jpeg","Images_textures/
underGround.jpeg");

    // ground=groundRandomgenerator(ground,renderer);
    SDL_Texture* groundTex = IMG_LoadTexture(renderer, "Images_textures/ground.jpeg");
    SDL_Texture* ugTex = IMG_LoadTexture(renderer, "Images_textures/underGround.jpeg");
    TTF_Font *font =TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf",100);
    std::vector<trapSpike> trap;
    std::vector<ground_Class> ground;
```

```

    Score score;
    GameOver gameOver;
    auto resetGameplay = [&]()
    {
        player1 = player();
        cammera_offSet = 0.5f;
        trap.clear();
        ground = ground_generator(renderer, groundTex, ugTex);
        groundRandomgenerator(renderer, groundTex, ugTex, ground, trap);
        // ground.push_back(temp);
        resetScore(score);
        gameOver.reset();
    };

    initializeScore(score);
    resetGameplay();
    bool quit=false;
    SDL_Event eventManager;
    Uint32 lastTime = SDL_GetTicks();
    int i=0;
    int count=0;
    const float targetFrameTime = 1000.0f / 60.0f; // 16.67 ms
    bool menu=false;
    while(!quit)
    {

        Uint32 frameStart = SDL_GetTicks();
        int screenW = currentScreenWidth;
        int screenH = currentScreenHeight;
        getScreenSize(renderer, screenW, screenH);
        // ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 6 - TRAP SYSTEM

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is traps.cpp, object.h. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

Trap Collision and Hitbox

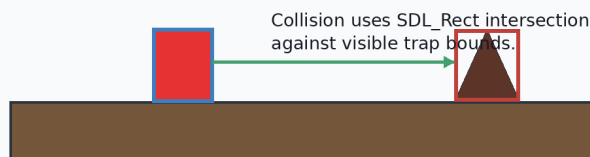


Figure 6.1 Trap

collision diagram

The visible triangle is approximated by a rectangle for SDL_HasIntersection collision checks.

Trap Probability and Attempts

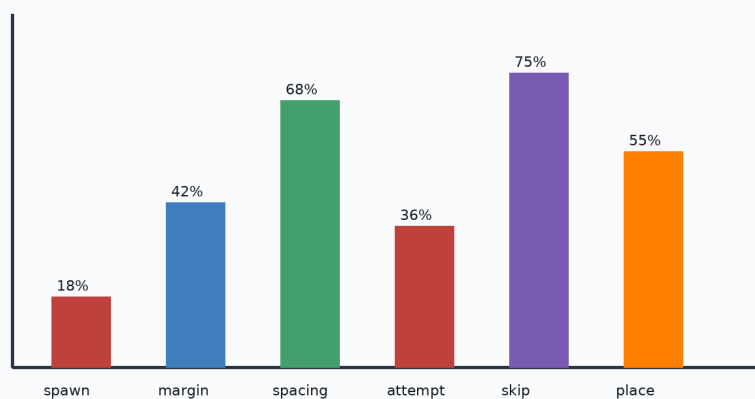


Figure 6.2 Spawn

probability chart

Each attempt has a probability gate, and the loop stops after maxAttempts to avoid runaway logic.

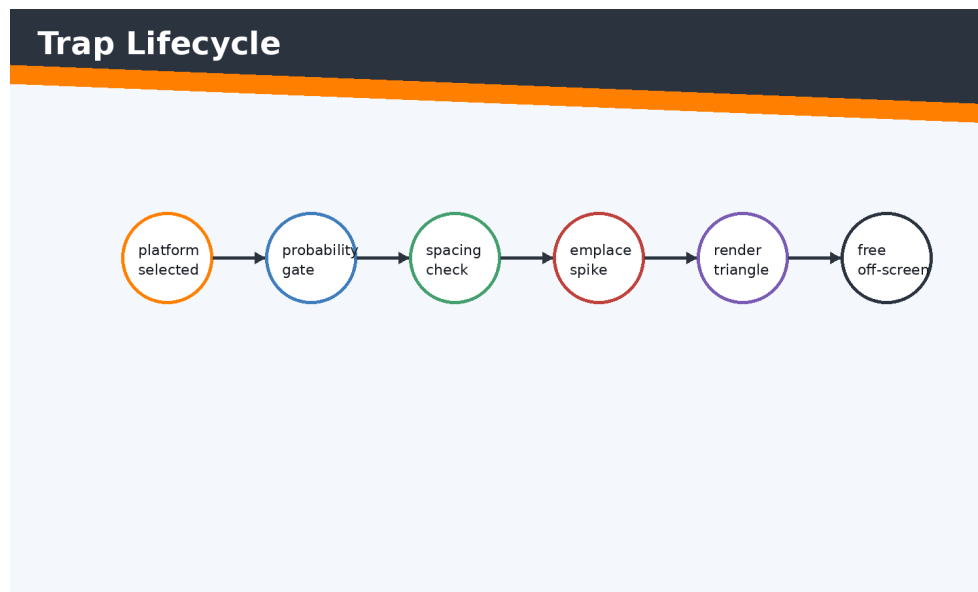


Figure 6.3 Trap

lifecycle flowchart

The trap system is linked to platform generation and off-screen cleanup.

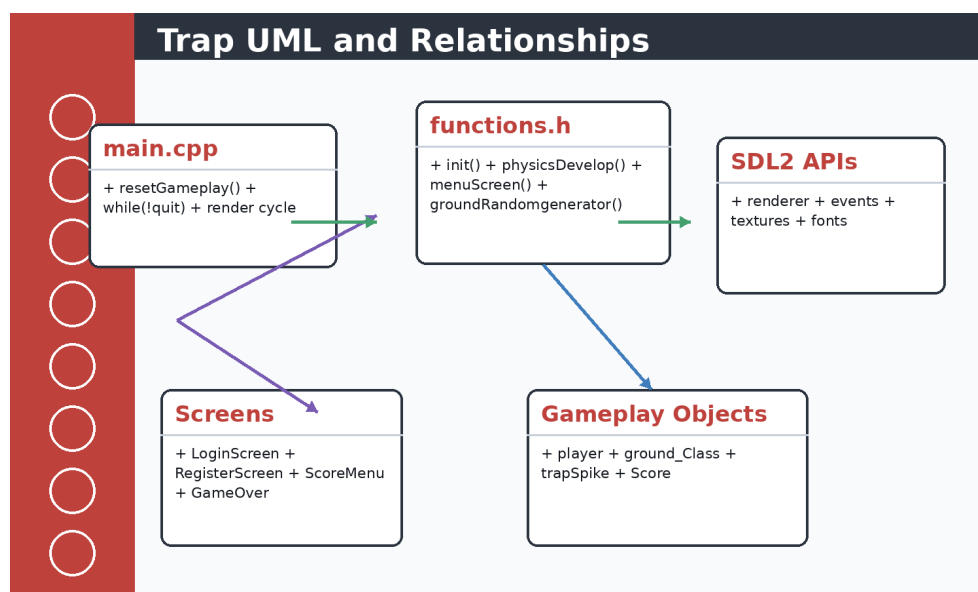


Figure 6.4 Trap UML

diagram

trapSpike inherits hitBox and depends on SDL2_gfx rendering functions.

Specific fixes: trap overlap is reduced with minTrapSpacing; probability is controlled by $\text{chances}(0,1000) < 100$; stack overflow/infinite loop risk is controlled by maxAttempts; SDL2_gfx int16 overflow streaks are reduced by visibility checks in trapSpike::render; trap color line issues are handled by drawing a filled triangle first and a darker outline second.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAll-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

Traps are spike triangles rendered with `SDL2_gfx`. Spawn probability is bounded by attempts, platform width margins, maximum trap count, and minimum spacing so traps do not overlap excessively or create infinite generation loops.

Code Excerpt: traps.cpp

```
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>
#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include<random>
#include "functions.h"

void trapTriangle(SDL_Renderer* renderer, std::vector<trapSpike>& trap,
std::vector<ground_Class>& ground, int i, int maxTraps)
{
    if (ground[i].first_ground || ground[i].trapCount >= maxTraps)
    {
        return;
    }

    int screenW = currentScreenWidth;
    int screenH = currentScreenHeight;
    getScreenSize(renderer, screenW, screenH);
    (void)screenH;
```

```
std::random_device rd;
std::mt19937 gen(rd());

// Add a safety limit to prevent infinite looping if conditions are hard to meet
int attempts = 0;
const int maxAttempts = 100;

while (attempts < maxAttempts)
{
    attempts++;
    std::uniform_int_distribution<> chances(0, 1000);

    // Probability check
    if (chances(gen) < 100) // Adjusted probability for more consistent generation
    {
        const int trapSize = scaledTrapSize();
        const int margin = std::max(100, trapSize * 2);
        if (ground[i].w <= margin * 2)
        {
            return;
        }

        std::uniform_int_distribution<> distp(ground[i].x + margin,
                                              (ground[i].x + ground[i].w) - margin);
        int spikeX = distp(gen);

        // Check for overlap with existing traps
        bool overlapping = false;
        const int minTrapSpacing = std::min(300, std::max(trapSize * 2, screenW / 10));
        for (const auto& existingTrap : trap)
        {
            if (spikeX - existingTrap.x <= minTrapSpacing &&
                spikeX - existingTrap.x >= -minTrapSpacing)
            {
                overlapping = true;
                break;
            }
        }

        if (!overlapping && ground[i].trapCount < maxTraps)
        {
            ground[i].trapCount++;
            trap.emplace_back(spikeX, ground[i].y, trapSize, trapSize);
            return; // Successfully added one trap
        }
    }
}

// ... excerpt trimmed for the report ...
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: object.h

```
#pragma once

#include <SDL2/SDL.h>
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_image.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_surface.h>
#include <iostream>
#include <vector>
#include <string>
#include <SDL2/SDL2_gfxPrimitives.h> //to import trigons
#include <SDL2/SDL_ttf.h>

extern int currentScreenWidth;
extern int currentScreenHeight;

inline int scaledGroundTileSize()
{
    if (currentScreenWidth >= 3840)
    {
        return 96;
    }
    if (currentScreenWidth >= 3200)
    {
        return 80;
    }
    if (currentScreenWidth >= 2400)
    {
        return 64;
    }
    return 50;
}

inline int scaledTrapSize()
{
    if (currentScreenWidth >= 3200)
    {
        return 88;
```

```

    }
    if (currentScreenWidth >= 2400)
    {
        return 68;
    }
    return 50;
}

inline int scaledPlayerSize()
{
    if (currentScreenWidth >= 3200)
    {
        return 88;
    }
    if (currentScreenWidth >= 2400)
    {
        return 68;
    }
    return 50;
}

class hitBox
{
public:
    int x, y, w, h;

    hitBox(int x, int y, int w, int h)
    {
        this->x = x;
        this->y = y;
        this->w = w;
        this->h = h;
    }
}

// ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent

values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 7 - MENU SYSTEM

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is menuScreen.cpp, object.h. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

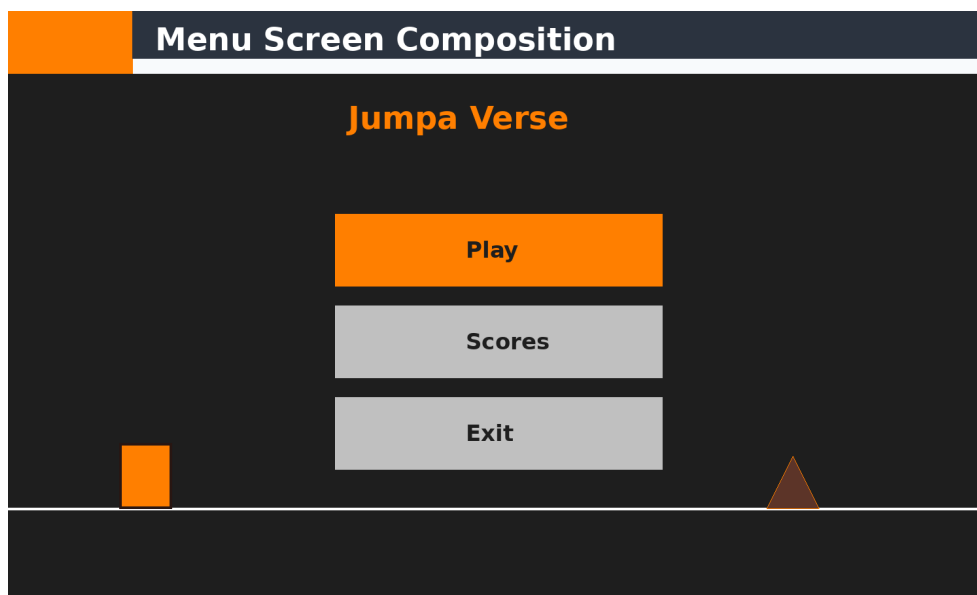


Figure 7.1 Menu UI

diagram

The visible screen uses SDL_RenderFillRect, text textures, line drawing, and SDL2_gfx triangles.

Button Hover State

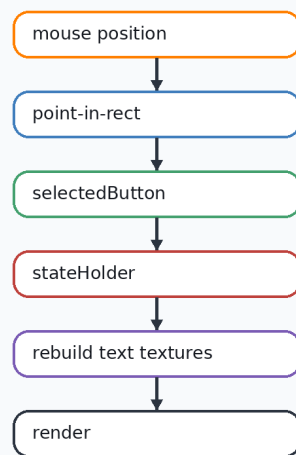


Figure 7.2 Hover effect

flow

Hover state changes selected colors and text textures.

Menu State Transitions

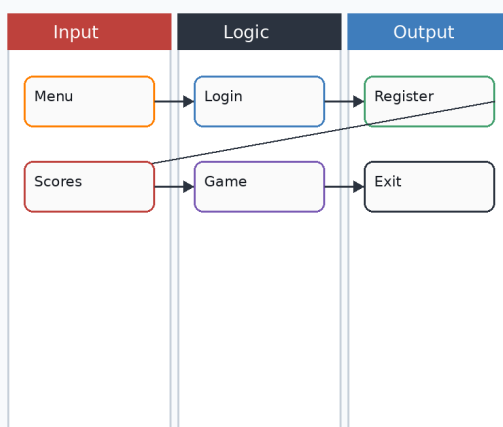


Figure 7.3 Menu state

diagram

The menu is the root navigation state for the application.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAI-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

The menu is an interactive SDL screen that draws the title, buttons, animated preview objects, and custom window controls. It handles keyboard selection, mouse hover, click actions, and transitions to login, score menu, or exit.

Code Excerpt: menuScreen.cpp

```
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>
#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include<SDL2/SDL2_gfxPrimitives.h>
#include<random>
#include "functions.h"

SDL_Color mainMenuColor::background = {30, 30, 30, 255};
SDL_Color mainMenuColor::buttonNormal = {70, 130, 180, 255};
SDL_Color mainMenuColor::buttonHover = {100, 160, 210, 255};
SDL_Color mainMenuColor::buttonActive = {40, 90, 140, 255};
SDL_Color mainMenuColor::text = {255, 255, 255, 255};

namespace
{
    bool openLoginForPlay(SDL_Renderer* renderer, TTF_Font* font)
    {
        LoginScreenAction action = runLoginScreen(renderer, font);
        if (action == LoginScreenAction::Quit)
        {
            exitb = true;
            return true;
        }

        return action == LoginScreenAction::LoginSuccess;
    }
}

//Run
```

```

bool menuScreen(SDL_Renderer* renderer, TTF_Font* font)
{
    optionBoxes playButton(740, 430, 500, 150);
    playButton.box_Initializer(renderer, font, "Play");
    SDL_Texture* titleTexture = nullptr;
    if (font)
    {
        SDL_Surface* titleSurface = TTF_RenderText_Blended(font, "Jumpa Verse", SDL_Color{255,
127, 0, 255});
        if (titleSurface)
        {
            titleTexture = SDL_CreateTextureFromSurface(renderer, titleSurface);
            SDL_FreeSurface(titleSurface);
        }
    }
    int screenW = currentScreenWidth;
    int screenH = currentScreenHeight;
    getScreenSize(renderer, screenW, screenH);
    SDL_Point start={0, screenH - 160};
    SDL_Point end={screenW, screenH - 160};

    bool running = true;
    SDL_Event event;
    int mousex,mousey;
    int selectedButton = 0;
    bool playSelected = true;
    bool scoresSelected = false;
    bool exitSelected = false;
    Uint32 lastFrameTime = SDL_GetTicks();
    float cubeVelocity = 0.0f;
    float cubeY = (float)(start.y - 45);
    float trapOffset = 0.0f;
    int lastSimulationLineY = start.y;

    // ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: object.h

```

#pragma once

```

```
#include <SDL2/SDL.h>
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_image.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_surface.h>
#include <iostream>
#include <vector>
#include <string>
#include <SDL2/SDL2_gfxPrimitives.h> //to import trigons
#include <SDL2/SDL_ttf.h>
```

```
extern int currentScreenWidth;
extern int currentScreenHeight;
```

```
inline int scaledGroundTileSize()
{
    if (currentScreenWidth >= 3840)
    {
        return 96;
    }
    if (currentScreenWidth >= 3200)
    {
        return 80;
    }
    if (currentScreenWidth >= 2400)
    {
        return 64;
    }
    return 50;
}
```

```
inline int scaledTrapSize()
{
    if (currentScreenWidth >= 3200)
    {
        return 88;
    }
    if (currentScreenWidth >= 2400)
    {
        return 68;
    }
    return 50;
}
```

```
inline int scaledPlayerSize()
{
    if (currentScreenWidth >= 3200)
    {
        return 88;
    }
    if (currentScreenWidth >= 2400)
    {
        return 68;
    }
    return 50;
}

class hitBox
{
public:
    int x, y, w, h;

    hitBox(int x, int y, int w, int h)
    {
        this->x = x;
        this->y = y;
        this->w = w;
        this->h = h;
    }
    // ... excerpt trimmed for the report ...
}
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 8 - PLAYER SYSTEM AND MOVEMENT

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is player.cpp, object.h. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

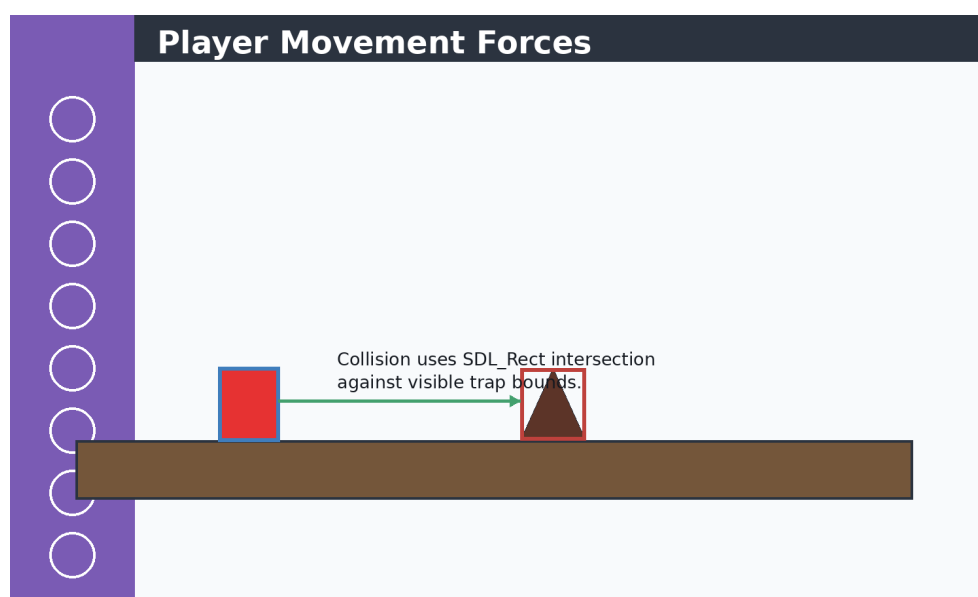


Figure 8.1 Movement

diagram

Jump velocity pushes up, gravity pulls down, and platform collision snaps the player onto the ground.

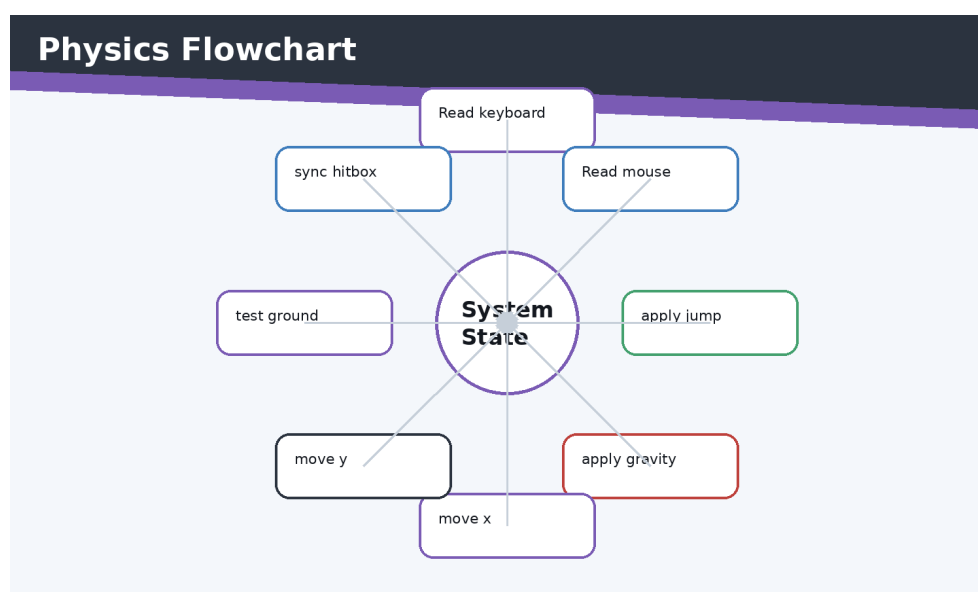


Figure 8.2 Physics

flowchart

physicsDevelop applies input before resolving vertical collision.

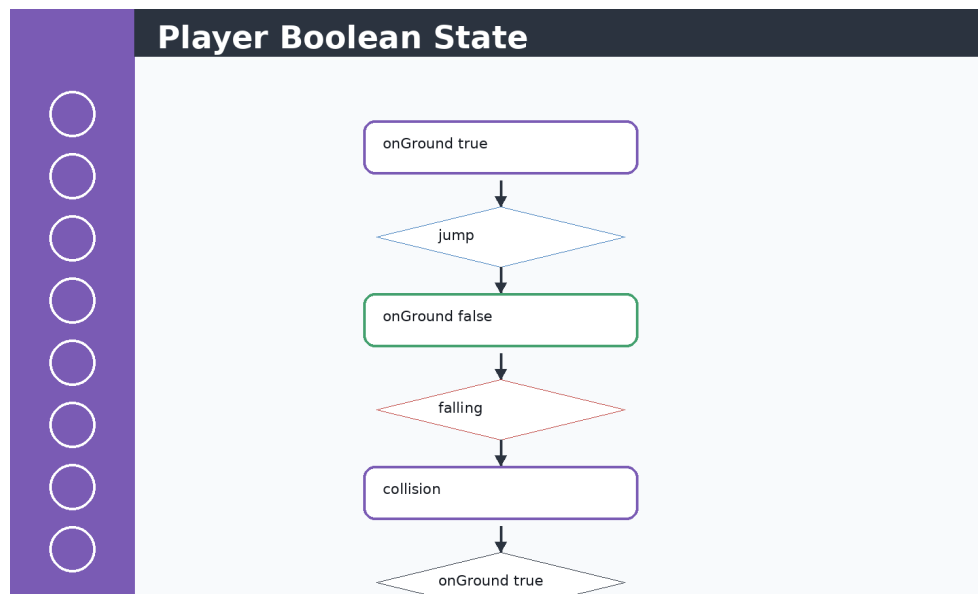


Figure 8.3 Player state

diagram

The state cycle is controlled by onGround and jumpVelocity.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAI-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

The player is a rectangle with velocity, gravity, jump power, and onGround state. Movement combines SDL keyboard state, mouse buttons, gravity integration, horizontal clamping, and top-only platform collision.

Code Excerpt: player.cpp

```
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_scancode.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
```

```
#include <iterator>
#include <ostream>
#include<vector>
#include<string.h>
#include<vector>
#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include "functions.h"
#include<stdlib.h>
#include<SDL2/SDL_ttf.h>
#include<limits>
int i=0;
void physicsDevelop(player& player1,
    std::vector<ground_Class>& g1,
    float deltaTime,
    const Uint8* keyboardState)
{
    Uint32 mouseState = SDL_GetMouseState(NULL, NULL);

    if (deltaTime <= 0.0f)
    {
        return;
    }

    bool jumpPressed = keyboardState[SDL_SCANCODE_SPACE] ||
        keyboardState[SDL_SCANCODE_W] ||
        keyboardState[SDL_SCANCODE_UP];

    bool characterDown=keyboardState[SDL_SCANCODE_LCTRL] ||
        keyboardState[SDL_SCANCODE_S] ||
        keyboardState[SDL_SCANCODE_DOWN];

    bool characterdash=keyboardState[SDL_SCANCODE_LSHIFT] ||
        keyboardState[SDL_SCANCODE_D] ||
        keyboardState[SDL_SCANCODE_RIGHT];
    bool backward=keyboardState[SDL_SCANCODE_A] ||
        keyboardState[SDL_SCANCODE_LEFT] ||
        keyboardState[SDL_SCANCODE_LALT];

    bool leftClick = mouseState & SDL_BUTTON(SDL_BUTTON_LEFT);
    bool rightClick = mouseState & SDL_BUTTON(SDL_BUTTON_RIGHT);
    bool middleClick= mouseState & SDL_BUTTON(SDL_BUTTON_MIDDLE);

    // if (mouseState & SDL_BUTTON(SDL_BUTTON_RIGHT))
```



```
// {
//     std::cout << "Right click detected\n";
// }
    if ((jumpPressed || leftClick)&& player1.onGround)
    {
        player1.jumpVelocity = player1.jumpPower;
        player1.onGround = false;
    }
    if (characterDown||rightClick)
    {
        player1.gravity+=2000;
    }

    const int playerMoveRight = std::min(9, std::max(5, currentScreenWidth / 360));
    const int playerMoveLeft = std::min(14, std::max(8, currentScreenWidth / 240));
    const int rightLimit = currentScreenWidth - player1.character.w;
    if(characterdash && (player1.character.x+player1.character.w)<currentScreenWidth)
    {
        player1.character.x += playerMoveRight;
        if (player1.character.x > rightLimit)
        {
            // ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: object.h

```
#pragma once

#include <SDL2/SDL.h>
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_image.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_surface.h>
#include <iostream>
#include <vector>
#include <string>
#include <SDL2/SDL2_gfxPrimitives.h> //to import trigons
#include<SDL2/SDL_ttf.h>

extern int currentScreenWidth;
extern int currentScreenHeight;

```

```
inline int scaledGroundTileSize()
{
    if (currentScreenWidth >= 3840)
    {
        return 96;
    }
    if (currentScreenWidth >= 3200)
    {
        return 80;
    }
    if (currentScreenWidth >= 2400)
    {
        return 64;
    }
    return 50;
}
```

```
inline int scaledTrapSize()
{
    if (currentScreenWidth >= 3200)
    {
        return 88;
    }
    if (currentScreenWidth >= 2400)
    {
        return 68;
    }
    return 50;
}
```

```
inline int scaledPlayerSize()
{
    if (currentScreenWidth >= 3200)
    {
        return 88;
    }
    if (currentScreenWidth >= 2400)
    {
        return 68;
    }
    return 50;
}
```

```
class hitBox
```

```
{
public:
    int x, y, w, h;

    hitBox(int x, int y, int w, int h)
    {
        this->x = x;
        this->y = y;
        this->w = w;
        this->h = h;
    }
    // ... excerpt trimmed for the report ...
}
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 9 - TEXT RENDERING ISSUES

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is score.cpp, loginScreen.cpp. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

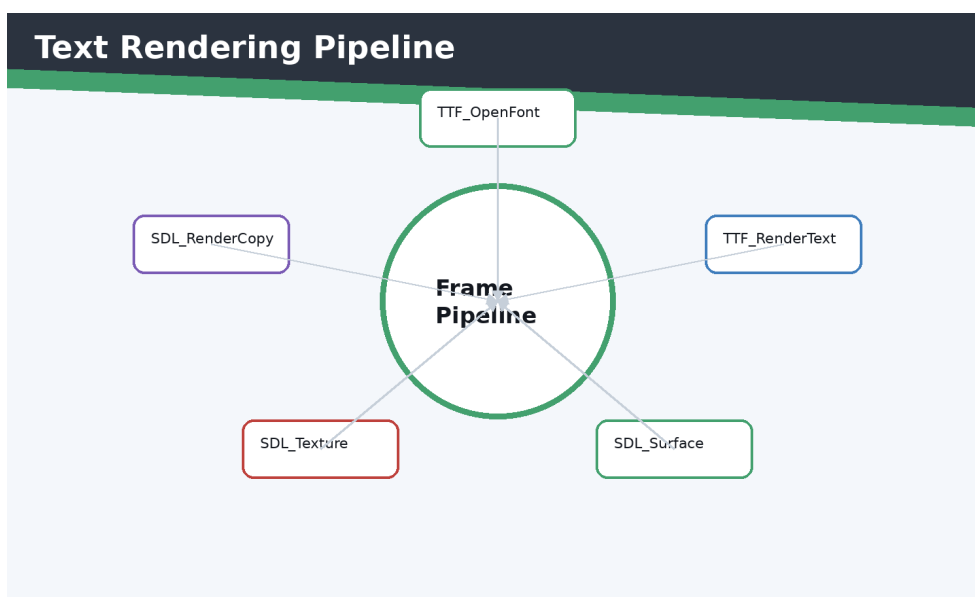


Figure 9.1 Text

rendering pipeline

Every visible text label is converted into texture data before it can be copied by the renderer.

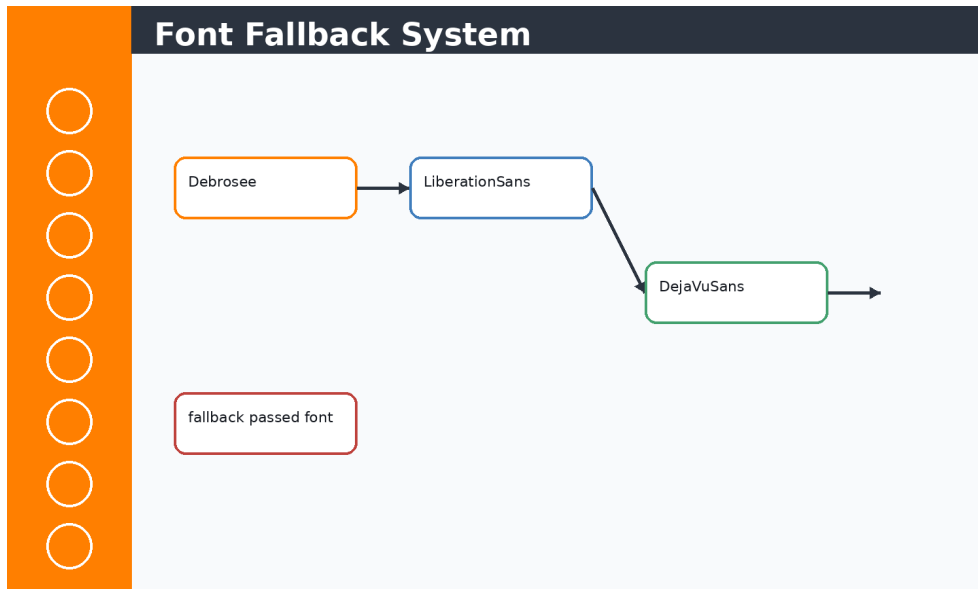


Figure 9.2 Font system

visual

The score system uses a digit fallback to solve integer rendering/readability issues.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAII-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

Text rendering in `SDL_ttf` is a two-step process: first rasterize a string to `SDL_Surface`, then upload it as `SDL_Texture`. The project uses font fallbacks for numeric score values because decorative fonts can make integers hard to read.

Code Excerpt: `score.cpp`

```
#include "object.h"
#include "functions.h"

#include <SDL2/SDL_ttf.h>
#include <SDL2/SDL.h>
#include <string>

Score::Score()
```

```
{
    initialize();
}

Score::~Score() = default;

void Score::initialize()
{
    reset();
}

void Score::update(float deltaTime, bool playerAlive)
{
    if (!playerAlive || deltaTime <= 0.0f)
    {
        return;
    }

    timeAccumulator += deltaTime;
    if (timeAccumulator >= 1.0f)
    {
        score += 1;
        timeAccumulator = 0.0f;
    }
}

void Score::render(SDL_Renderer* renderer, TTF_Font* font)
{
    if (!renderer)
    {
        return;
    }

    static TTF_Font* labelFont = nullptr;
    if (!labelFont)
    {
        labelFont = TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf", 26);
        if (!labelFont)
        {
            labelFont = font;
        }
    }

    static TTF_Font* digitFont = nullptr;
    if (!digitFont)
```

```

    {
        digitFont = TTF_OpenFont("/usr/share/fonts/liberation/LiberationSans-Bold.ttf", 26);
        if (!digitFont)
        {
            digitFont = TTF_OpenFont("/usr/share/fonts/TTF/DejaVuSans-Bold.ttf", 26);
        }
    }

    if (!labelFont || !digitFont)
    {
        return;
    }

    int screenW = currentScreenWidth;
    int screenH = currentScreenHeight;
    getScreenSize(renderer, screenW, screenH);

    // ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: loginScreen.cpp

```

#include "object.h"
#include "functions.h"

#include <SDL2/SDL.h>
#include <SDL2/SDL_ttf.h>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

namespace
{
    const std::string playerFile = "player.csv";

    bool pointInside(const SDL_Rect& rect, int x, int y)
    {
        return x >= rect.x && x <= rect.x + rect.w &&
            y >= rect.y && y <= rect.y + rect.h;
    }
}

```

```
}
```

```
TTF_Font* openFont(int size, TTF_Font* fallback)
```

```
{
```

```
    TTF_Font* font = TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf", size);
```

```
    return font ? font : fallback;
```

```
}
```

```
void drawText(SDL_Renderer* renderer,
```

```
              TTF_Font* font,
```

```
              const std::string& text,
```

```
              SDL_Color color,
```

```
              SDL_Rect bounds,
```

```
              bool center = true)
```

```
{
```

```
    if (!renderer || !font || text.empty())
```

```
    {
```

```
        return;
```

```
    }
```

```
    SDL_Surface* surface = TTF_RenderUTF8_Blended(font, text.c_str(), color);
```

```
    if (!surface)
```

```
    {
```

```
        return;
```

```
    }
```

```
    SDL_Texture* texture = SDL_CreateTextureFromSurface(renderer, surface);
```

```
    SDL_FreeSurface(surface);
```

```
    if (!texture)
```

```
    {
```

```
        return;
```

```
    }
```

```
    int textW = 0;
```

```
    int textH = 0;
```

```
    SDL_QueryTexture(texture, nullptr, nullptr, &textW, &textH);
```

```
    float scaleW = textW > 0 ? (float)bounds.w / (float)textW : 1.0f;
```

```
    float scaleH = textH > 0 ? (float)bounds.h / (float)textH : 1.0f;
```

```
    float scale = scaleW < scaleH ? scaleW : scaleH;
```

```
    if (scale > 1.0f)
```

```
    {
```

```
        scale = 1.0f;
```

```
    }
```

```
    int drawW = (int)(textW * scale);
```



```
int drawH = (int)(textH * scale);
SDL_Rect target = {
    center ? bounds.x + (bounds.w - drawW) / 2 : bounds.x,
    bounds.y + (bounds.h - drawH) / 2,
    drawW,
    drawH
};
// ... excerpt trimmed for the report ...
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 10 - ERROR HANDLING SYSTEM

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is Error_Handlers.cpp, main.cpp. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

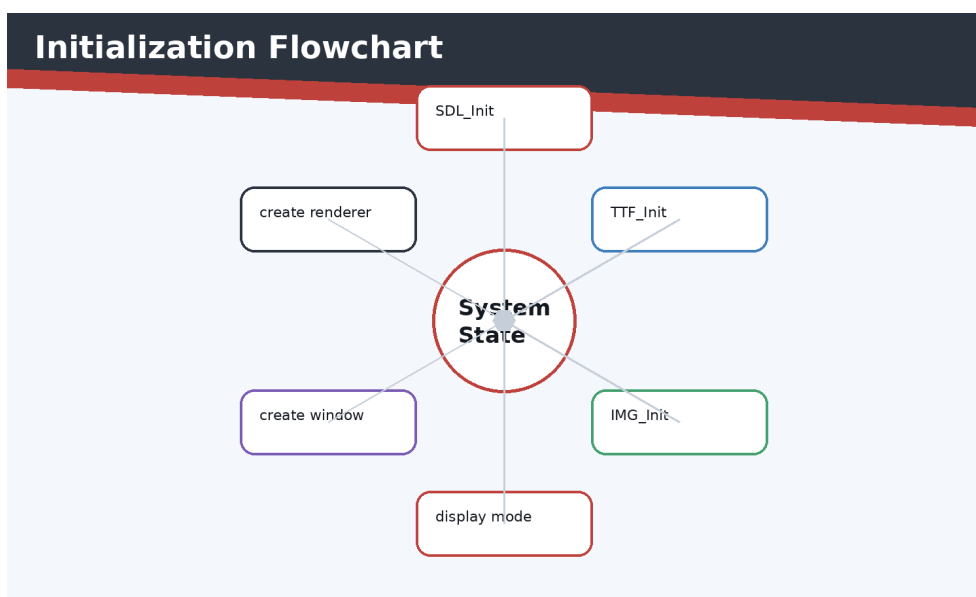


Figure 10.1

Initialization flowchart

Each initialization step must succeed before gameplay begins.

Error Handling Branches

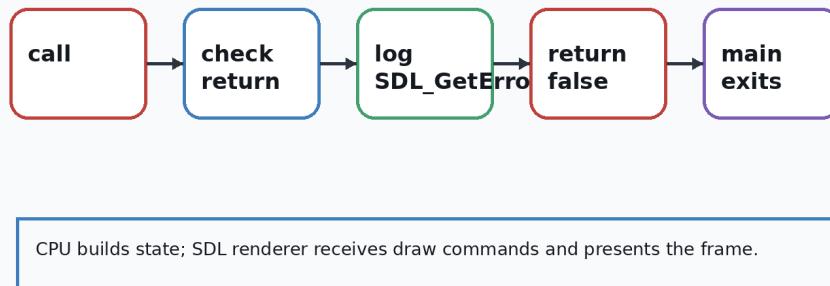


Figure 10.2 Error

handling diagram

The system prevents null window or renderer usage.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAII-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

Initialization is defensive: SDL, `SDL_ttf`, `SDL_image`, window creation, and renderer creation are each checked. Failure returns false and `main.cpp` exits early instead of entering an invalid render loop.

Code Excerpt: `Error_Handlers.cpp`

```
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include<vector>
#include<string.h>
#include<vector>
#include<algorithm> //For basic algorithm like sorting
#include"object.h"
```

```
#include "functions.h"
#include <SDL2/SDL_image.h>

bool init(SDL_Window*& Window, SDL_Renderer *&renderer); //Check
void close(SDL_Window* window, SDL_Renderer* renderer);

int currentScreenWidth = GAME_WIDTH;
int currentScreenHeight = GAME_HEIGHT;

bool init(SDL_Window*& Window, SDL_Renderer *&renderer)
{
    if(SDL_Init(SDL_INIT_VIDEO)<0)
    {
        std::cerr<<"SDL couldnt be initialized! \t\t\t Error Type:\t"<<SDL_GetError()<<std::endl;
        return false;
    }
    if(TTF_Init()<0)
    {
        std::cout<<"TTF NOT FOUND"<<std::endl;
        return false;
    }

    if (!(IMG_Init(IMG_INIT_JPG) & IMG_INIT_JPG))
    {
        std::cerr << "SDL_image could not initialize! Error: " << IMG_GetError() << std::endl;
        return false;
    }

    SDL_DisplayMode displayMode;
    if (SDL_GetCurrentDisplayMode(0, &displayMode) == 0)
    {
        currentScreenWidth = displayMode.w;
        currentScreenHeight = displayMode.h;
    }

    Window = SDL_CreateWindow("Jumpa Verse",
                               SDL_WINDOWPOS_CENTERED,
                               SDL_WINDOWPOS_CENTERED,
                               currentScreenWidth,
                               currentScreenHeight,
                               SDL_WINDOW_SHOWN | SDL_WINDOW_RESIZABLE |
SDL_WINDOW_FULLSCREEN_DESKTOP);
```

```

        if(!Window)
        {
            std::cerr<<"SDL WINDOW COULDNT START \t\t\tERROR TYPE:\t"<<SDL_GetError()<<std::endl;
            return false;
        }
        renderer=SDL_CreateRenderer(Window,-1,SDL_RENDERER_ACCELERATED);

        if(!renderer)
        {
            std::cerr<<"RENDERER COULDNT LOAD!\t\t\tSDL ERROR:\t"<<SDL_GetError()<<std::endl;
            return false;
        }

        SDL_SetWindowMinimumSize(Window, 960, 540);

        return true;
// ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: main.cpp

```

#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>
#include<vector>
#include<algorithm>//For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include "functions.h"
#include<stdlib.h>
#include<SDL2/SDL_ttf.h>
float cammeraDash=0;
bool exitb = false;
int main()
{

```

```
    SDL_Window* window=nullptr;
    SDL_Renderer* renderer=nullptr;

    player player1;

    if(!init(window,renderer))
    {
        std::cerr<<"FAILED TO START "<<std::endl;
        return -1;
    }

    float cammera_offSet=0.5f; //no 2d camera in sdl2 so we create an illusion
    // ground_Class
    ground(100,900,500,100,renderer,"Images_textures/ground.jpeg","Images_textures/
underGround.jpeg");

    // ground=groundRandomgenerator(ground,renderer);
    SDL_Texture* groundTex = IMG_LoadTexture(renderer, "Images_textures/ground.jpeg");
    SDL_Texture* ugTex = IMG_LoadTexture(renderer, "Images_textures/underGround.jpeg");
    TTF_Font *font =TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf",100);
    std::vector<trapSpike> trap;
    std::vector<ground_Class> ground;
    Score score;
    GameOver gameOver;
    auto resetGameplay = [&]()
    {
        player1 = player();
        cammera_offSet = 0.5f;
        trap.clear();
        ground = ground_generator(renderer, groundTex, ugTex);
        groundRandomgenerator(renderer, groundTex, ugTex, ground, trap);
        // ground.push_back(temp);
        resetScore(score);
        gameOver.reset();
    };

    initializeScore(score);
    resetGameplay();
    bool quit=false;
    SDL_Event eventManager;
    Uint32 lastTime = SDL_GetTicks();
    int i=0;
    int count=0;
```

```
const float targetFrameTime = 1000.0f / 60.0f; // 16.67 ms
bool menu=false;
while(!quit)
{

    Uint32 frameStart = SDL_GetTicks();
    int screenW = currentScreenWidth;
    int screenH = currentScreenHeight;
    getScreenSize(renderer, screenW, screenH);
    // ... excerpt trimmed for the report ...
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 11 - LOGIN SCREEN AND GAME OVER SCREEN

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is loginScreen.cpp, register.cpp, gameover.cpp. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

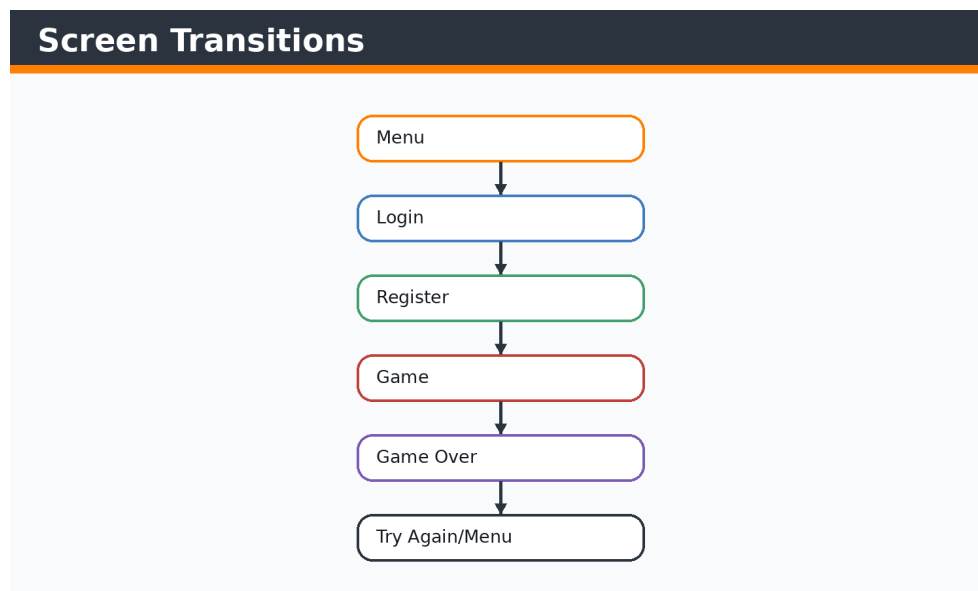


Figure 11.1 Screen

transition diagram

Screen action enums make transitions explicit and readable.

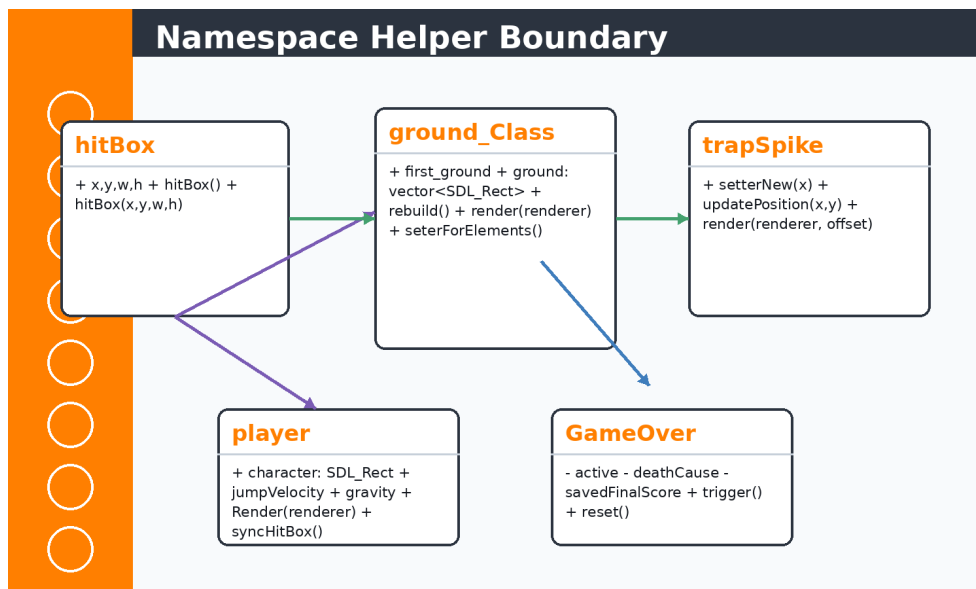


Figure 11.2

Namespace diagram

Anonymous namespaces keep helper functions private to their translation units.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAII-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

Login, registration, scores, and game over are separate screen loops. They use namespaces for helper functions, enums for actions, text input events for form fields, and CSV files for persistent player data.

Code Excerpt: loginScreen.cpp

```

#include "object.h"
#include "functions.h"

#include <SDL2/SDL.h>
#include <SDL2/SDL_ttf.h>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
  
```

```
namespace
{
    const std::string playerFile = "player.csv";

    bool pointInside(const SDL_Rect& rect, int x, int y)
    {
        return x >= rect.x && x <= rect.x + rect.w &&
            y >= rect.y && y <= rect.y + rect.h;
    }

    TTF_Font* openFont(int size, TTF_Font* fallback)
    {
        TTF_Font* font = TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf", size);
        return font ? font : fallback;
    }

    void drawText(SDL_Renderer* renderer,
                  TTF_Font* font,
                  const std::string& text,
                  SDL_Color color,
                  SDL_Rect bounds,
                  bool center = true)
    {
        if (!renderer || !font || text.empty())
        {
            return;
        }

        SDL_Surface* surface = TTF_RenderUTF8_Blended(font, text.c_str(), color);
        if (!surface)
        {
            return;
        }

        SDL_Texture* texture = SDL_CreateTextureFromSurface(renderer, surface);
        SDL_FreeSurface(surface);
        if (!texture)
        {
            return;
        }

        int textW = 0;
        int textH = 0;
        SDL_QueryTexture(texture, nullptr, nullptr, &textW, &textH);
```

```

float scaleW = textW > 0 ? (float)bounds.w / (float)textW : 1.0f;
float scaleH = textH > 0 ? (float)bounds.h / (float)textH : 1.0f;
float scale = scaleW < scaleH ? scaleW : scaleH;
if (scale > 1.0f)
{
    scale = 1.0f;
}

int drawW = (int)(textW * scale);
int drawH = (int)(textH * scale);
SDL_Rect target = {
    center ? bounds.x + (bounds.w - drawW) / 2 : bounds.x,
    bounds.y + (bounds.h - drawH) / 2,
    drawW,
    drawH
};
// ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: register.cpp

```

#include "object.h"
#include "functions.h"

#include <SDL2/SDL.h>
#include <SDL2/SDL_ttf.h>
#include <fstream>
#include <sstream>
#include <string>

namespace
{
    const std::string playerFile = "player.csv";

    bool pointInside(const SDL_Rect& rect, int x, int y)
    {
        return x >= rect.x && x <= rect.x + rect.w &&
            y >= rect.y && y <= rect.y + rect.h;
    }
}

```

```
TTF_Font* openFont(int size, TTF_Font* fallback)
{
    TTF_Font* font = TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf", size);
    return font ? font : fallback;
}

void drawText(SDL_Renderer* renderer,
              TTF_Font* font,
              const std::string& text,
              SDL_Color color,
              SDL_Rect bounds,
              bool center = true)
{
    if (!renderer || !font || text.empty())
    {
        return;
    }

    SDL_Surface* surface = TTF_RenderUTF8_Blended(font, text.c_str(), color);
    if (!surface)
    {
        return;
    }

    SDL_Texture* texture = SDL_CreateTextureFromSurface(renderer, surface);
    SDL_FreeSurface(surface);
    if (!texture)
    {
        return;
    }

    int textW = 0;
    int textH = 0;
    SDL_QueryTexture(texture, nullptr, nullptr, &textW, &textH);
    float scaleW = textW > 0 ? (float)bounds.w / (float)textW : 1.0f;
    float scaleH = textH > 0 ? (float)bounds.h / (float)textH : 1.0f;
    float scale = scaleW < scaleH ? scaleW : scaleH;
    if (scale > 1.0f)
    {
        scale = 1.0f;
    }

    int drawW = (int)(textW * scale);
    int drawH = (int)(textH * scale);
    SDL_Rect target = {
```

```
        center ? bounds.x + (bounds.w - drawW) / 2 : bounds.x,  
        bounds.y + (bounds.h - drawH) / 2,  
        drawW,  
        drawH  
    };
```

```
// ... excerpt trimmed for the report ...
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 12 - FILE HANDLING

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is loginScreen.cpp, register.cpp, gameOver.cpp, scoreMenu.cpp. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

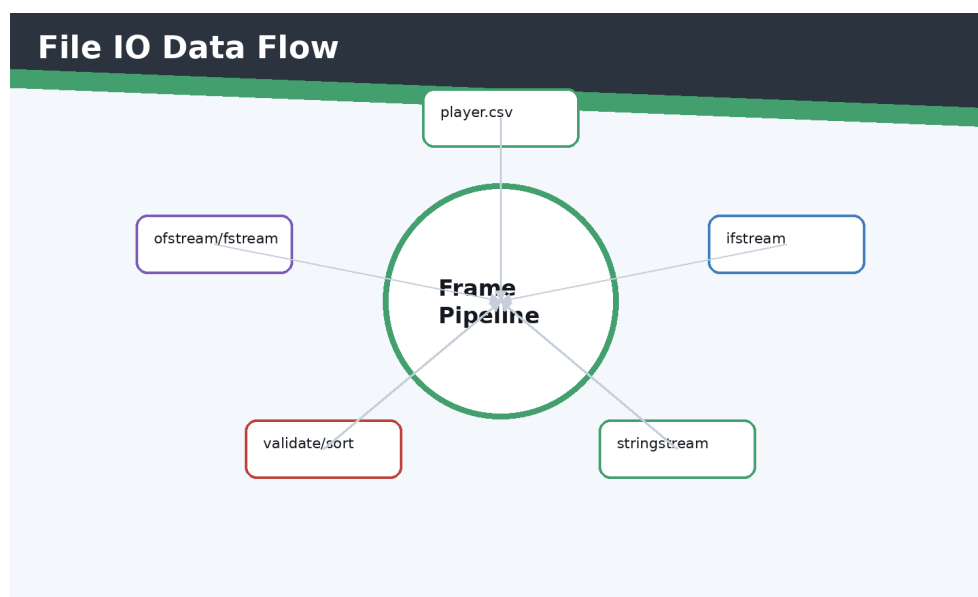


Figure 12.1 File IO

diagram

The player file stores username,password,score history in comma-separated lines.

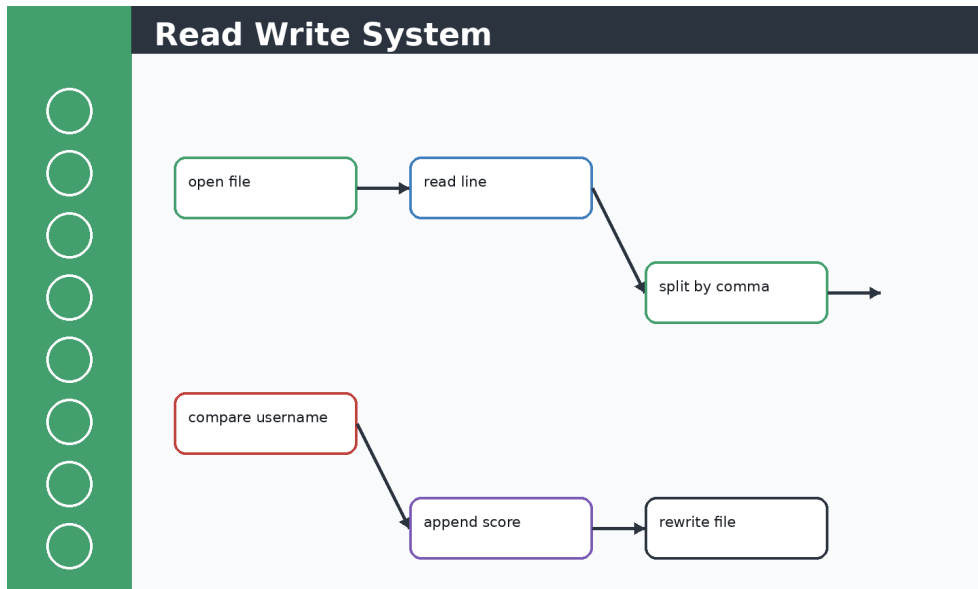


Figure 12.2 Data flow

visual

The game over screen appends scores to the logged-in player row.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAII-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

CSV persistence is intentionally beginner-friendly. `ifstream` reads existing players and scores, `ofstream` appends final scores, and `fstream` appends registered players. String streams parse comma-separated records.

Code Excerpt: `loginScreen.cpp`

```
#include "object.h"
#include "functions.h"

#include <SDL2/SDL.h>
#include <SDL2/SDL_ttf.h>
#include <fstream>
#include <sstream>
#include <string>
```

```
#include <vector>

namespace
{
    const std::string playerFile = "player.csv";

    bool pointInside(const SDL_Rect& rect, int x, int y)
    {
        return x >= rect.x && x <= rect.x + rect.w &&
            y >= rect.y && y <= rect.y + rect.h;
    }

    TTF_Font* openFont(int size, TTF_Font* fallback)
    {
        TTF_Font* font = TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf", size);
        return font ? font : fallback;
    }

    void drawText(SDL_Renderer* renderer,
        TTF_Font* font,
        const std::string& text,
        SDL_Color color,
        SDL_Rect bounds,
        bool center = true)
    {
        if (!renderer || !font || text.empty())
        {
            return;
        }

        SDL_Surface* surface = TTF_RenderUTF8_Blended(font, text.c_str(), color);
        if (!surface)
        {
            return;
        }

        SDL_Texture* texture = SDL_CreateTextureFromSurface(renderer, surface);
        SDL_FreeSurface(surface);
        if (!texture)
        {
            return;
        }

        int textW = 0;
        int textH = 0;
```



```

        SDL_QueryTexture(texture, nullptr, nullptr, &textW, &textH);
        float scaleW = textW > 0 ? (float)bounds.w / (float)textW : 1.0f;
        float scaleH = textH > 0 ? (float)bounds.h / (float)textH : 1.0f;
        float scale = scaleW < scaleH ? scaleW : scaleH;
        if (scale > 1.0f)
        {
            scale = 1.0f;
        }

        int drawW = (int)(textW * scale);
        int drawH = (int)(textH * scale);
        SDL_Rect target = {
            center ? bounds.x + (bounds.w - drawW) / 2 : bounds.x,
            bounds.y + (bounds.h - drawH) / 2,
            drawW,
            drawH
        };
    // ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: register.cpp

```

#include "object.h"
#include "functions.h"

#include <SDL2/SDL.h>
#include <SDL2/SDL_ttf.h>
#include <fstream>
#include <sstream>
#include <string>

namespace
{
    const std::string playerFile = "player.csv";

    bool pointInside(const SDL_Rect& rect, int x, int y)
    {
        return x >= rect.x && x <= rect.x + rect.w &&
            y >= rect.y && y <= rect.y + rect.h;
    }
}

```

```
TTF_Font* openFont(int size, TTF_Font* fallback)
{
    TTF_Font* font = TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf", size);
    return font ? font : fallback;
}
```

```
void drawText(SDL_Renderer* renderer,
              TTF_Font* font,
              const std::string& text,
              SDL_Color color,
              SDL_Rect bounds,
              bool center = true)
{
    if (!renderer || !font || text.empty())
    {
        return;
    }
```

```
    SDL_Surface* surface = TTF_RenderUTF8_Blended(font, text.c_str(), color);
    if (!surface)
    {
        return;
    }
```

```
    SDL_Texture* texture = SDL_CreateTextureFromSurface(renderer, surface);
    SDL_FreeSurface(surface);
    if (!texture)
    {
        return;
    }
```

```
    int textW = 0;
    int textH = 0;
    SDL_QueryTexture(texture, nullptr, nullptr, &textW, &textH);
    float scaleW = textW > 0 ? (float)bounds.w / (float)textW : 1.0f;
    float scaleH = textH > 0 ? (float)bounds.h / (float)textH : 1.0f;
    float scale = scaleW < scaleH ? scaleW : scaleH;
    if (scale > 1.0f)
    {
        scale = 1.0f;
    }
```

```
    int drawW = (int)(textW * scale);
    int drawH = (int)(textH * scale);
```

```
    SDL_Rect target = {
        center ? bounds.x + (bounds.w - drawW) / 2 : bounds.x,
        bounds.y + (bounds.h - drawH) / 2,
        drawW,
        drawH
    };

    // ... excerpt trimmed for the report ...
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 13 - LOGIN/REGISTRATION

ANIMATIONS

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is loginScreen.cpp, register.cpp. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

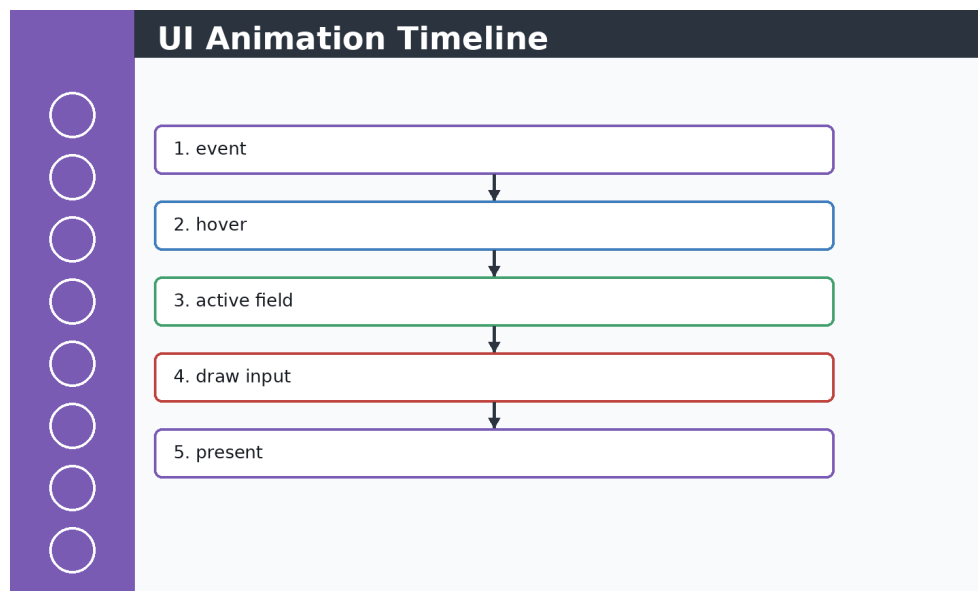


Figure 13.1 Animation

timeline

Interactive feedback is produced by redrawing selected states each frame.

Frame Progression

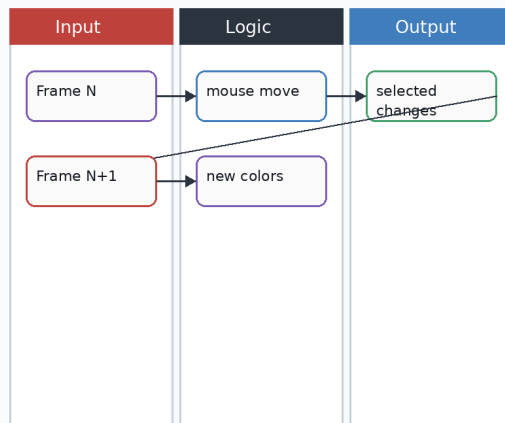


Figure 13.2 Frame

progression visual

The user sees animation because the render loop keeps presenting updated states.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAII-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

The account screens use frame-by-frame redraws, hover highlights, active input borders, and message transitions. The animation model is simple: each loop handles events, updates visual state, clears, draws, presents, and delays.

Code Excerpt: loginScreen.cpp

```
#include "object.h"
#include "functions.h"

#include <SDL2/SDL.h>
#include <SDL2/SDL_ttf.h>
#include <fstream>
#include <sstream>
#include <string>
```

```
#include <vector>

namespace
{
    const std::string playerFile = "player.csv";

    bool pointInside(const SDL_Rect& rect, int x, int y)
    {
        return x >= rect.x && x <= rect.x + rect.w &&
            y >= rect.y && y <= rect.y + rect.h;
    }

    TTF_Font* openFont(int size, TTF_Font* fallback)
    {
        TTF_Font* font = TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf", size);
        return font ? font : fallback;
    }

    void drawText(SDL_Renderer* renderer,
        TTF_Font* font,
        const std::string& text,
        SDL_Color color,
        SDL_Rect bounds,
        bool center = true)
    {
        if (!renderer || !font || text.empty())
        {
            return;
        }

        SDL_Surface* surface = TTF_RenderUTF8_Blended(font, text.c_str(), color);
        if (!surface)
        {
            return;
        }

        SDL_Texture* texture = SDL_CreateTextureFromSurface(renderer, surface);
        SDL_FreeSurface(surface);
        if (!texture)
        {
            return;
        }

        int textW = 0;
        int textH = 0;
```

```

        SDL_QueryTexture(texture, nullptr, nullptr, &textW, &textH);
        float scaleW = textW > 0 ? (float)bounds.w / (float)textW : 1.0f;
        float scaleH = textH > 0 ? (float)bounds.h / (float)textH : 1.0f;
        float scale = scaleW < scaleH ? scaleW : scaleH;
        if (scale > 1.0f)
        {
            scale = 1.0f;
        }

        int drawW = (int)(textW * scale);
        int drawH = (int)(textH * scale);
        SDL_Rect target = {
            center ? bounds.x + (bounds.w - drawW) / 2 : bounds.x,
            bounds.y + (bounds.h - drawH) / 2,
            drawW,
            drawH
        };
    // ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: register.cpp

```

#include "object.h"
#include "functions.h"

#include <SDL2/SDL.h>
#include <SDL2/SDL_ttf.h>
#include <fstream>
#include <sstream>
#include <string>

namespace
{
    const std::string playerFile = "player.csv";

    bool pointInside(const SDL_Rect& rect, int x, int y)
    {
        return x >= rect.x && x <= rect.x + rect.w &&
            y >= rect.y && y <= rect.y + rect.h;
    }
}

```

```
TTF_Font* openFont(int size, TTF_Font* fallback)
{
    TTF_Font* font = TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf", size);
    return font ? font : fallback;
}
```

```
void drawText(SDL_Renderer* renderer,
              TTF_Font* font,
              const std::string& text,
              SDL_Color color,
              SDL_Rect bounds,
              bool center = true)
{
    if (!renderer || !font || text.empty())
    {
        return;
    }
```

```
    SDL_Surface* surface = TTF_RenderUTF8_Blended(font, text.c_str(), color);
    if (!surface)
    {
        return;
    }
```

```
    SDL_Texture* texture = SDL_CreateTextureFromSurface(renderer, surface);
    SDL_FreeSurface(surface);
    if (!texture)
    {
        return;
    }
```

```
    int textW = 0;
    int textH = 0;
    SDL_QueryTexture(texture, nullptr, nullptr, &textW, &textH);
    float scaleW = textW > 0 ? (float)bounds.w / (float)textW : 1.0f;
    float scaleH = textH > 0 ? (float)bounds.h / (float)textH : 1.0f;
    float scale = scaleW < scaleH ? scaleW : scaleH;
    if (scale > 1.0f)
    {
        scale = 1.0f;
    }
```

```
    int drawW = (int)(textW * scale);
    int drawH = (int)(textH * scale);
```



```
    SDL_Rect target = {  
        center ? bounds.x + (bounds.w - drawW) / 2 : bounds.x,  
        bounds.y + (bounds.h - drawH) / 2,  
        drawW,  
        drawH  
    };  
  
    // ... excerpt trimmed for the report ...
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 14 - COMPLETE GAME FLOW

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is main.cpp, functions.h. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

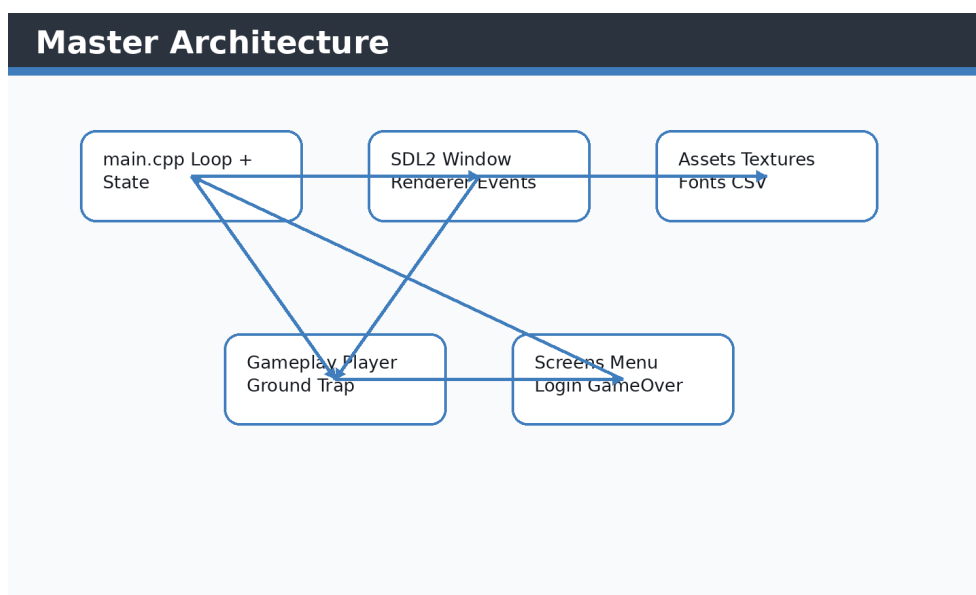


Figure 14.1 Master

architecture diagram

The full application is a set of screen and gameplay loops coordinated by main.cpp.

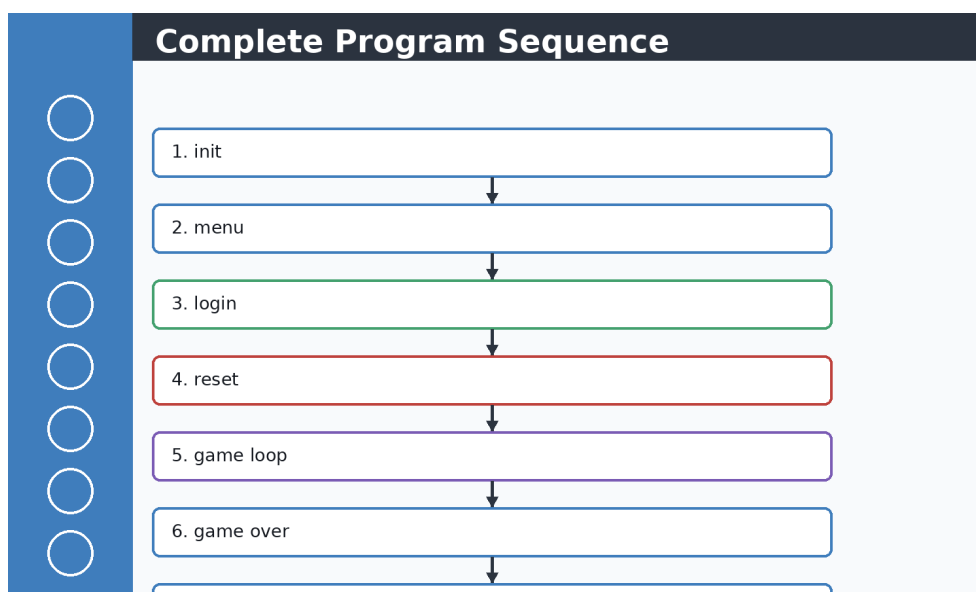


Figure 14.2 Full

sequence diagram

This is the end-to-end program route from startup to shutdown.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAII-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

The complete system starts at initialization, enters menu navigation, resets gameplay state, processes the real-time loop, saves death results, and returns to menu or restarts based on the selected action.

Code Excerpt: main.cpp

```
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>
#include<vector>
#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include "functions.h"
#include<stdlib.h>
#include<SDL2/SDL_ttf.h>
float cammeraDash=0;
bool exitb = false;
int main()
{
    SDL_Window* window=nullptr;
    SDL_Renderer* renderer=nullptr;

    player player1;
```

```

    if(!init(window,renderer))
    {
        std::cerr<<"FAILED TO START "<<std::endl;
        return -1;
    }

    float cammera_offSet=0.5f; //no 2d camera in sdl2 so we create an illusion
    // ground_Class
    ground(100,900,500,100,renderer,"Images_textures/ground.jpeg","Images_textures/
    underGround.jpeg");

    // ground=groundRandomgenerator(ground,renderer);
    SDL_Texture* groundTex = IMG_LoadTexture(renderer, "Images_textures/ground.jpeg");
    SDL_Texture* ugTex = IMG_LoadTexture(renderer, "Images_textures/underGround.jpeg");
    TTF_Font *font =TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf",100);
    std::vector<trapSpike> trap;
    std::vector<ground_Class> ground;
    Score score;
    GameOver gameOver;
    auto resetGameplay = [&]()
    {
        player1 = player();
        cammera_offSet = 0.5f;
        trap.clear();
        ground = ground_generator(renderer, groundTex, ugTex);
        groundRandomgenerator(renderer, groundTex, ugTex, ground, trap);
        // ground.push_back(temp);
        resetScore(score);
        gameOver.reset();
    };

    initializeScore(score);
    resetGameplay();
    bool quit=false;
    SDL_Event eventManager;
    Uint32 lastTime = SDL_GetTicks();
    int i=0;
    int count=0;
    const float targetFrameTime = 1000.0f / 60.0f; // 16.67 ms
    bool menu=false;
    while(!quit)
    {
        Uint32 frameStart = SDL_GetTicks();

```

```
int screenW = currentScreenWidth;
int screenH = currentScreenHeight;
getScreenSize(renderer, screenW, screenH);
// ... excerpt trimmed for the report ...
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: functions.h

```
#pragma once
#include <SDL2/SDL.h>
#include <vector>
#include <SDL2/SDL_ttf.h>
#include <string>
#include <cctype>

inline std::string toUpperUsername(const std::string& username)
{
    std::string upperUsername = username;
    for (char& character : upperUsername)
    {
        character = (char)std::toupper((unsigned char)character);
    }
    return upperUsername;
}

extern bool exitb;
extern float cammeraDash;
extern std::string currentPlayerUsername;

constexpr int GAME_WIDTH = 1980;
constexpr int GAME_HEIGHT = 1260;
extern int currentScreenWidth;
extern int currentScreenHeight;

class ground_Class;
class trapSpike;
class player;
class Score;
class GameOver;
enum class GameOverAction;
```

```
enum class DeathCause;
enum class LoginScreenAction;
enum class RegisterScreenAction;
bool init(SDL_Window*& Window, SDL_Renderer*& renderer);
void close(SDL_Window* window, SDL_Renderer* renderer);
void getScreenSize(SDL_Renderer* renderer, int& screenW, int& screenH);
bool handleWindowControlEvent(SDL_Renderer* renderer, const SDL_Event& event);
void renderWindowControls(SDL_Renderer* renderer);

std::vector<ground_Class> ground_generator(SDL_Renderer* renderer,SDL_Texture*
groundTex,SDL_Texture* ugTex);

// ground_Class groundRandomgenerator(SDL_Renderer* renderer,SDL_Texture* groundTex,SDL_Texture*
ugTex,std::vector<ground_Class> &prevGround,std::vector<trapSpike>& trap);

void groundRandomgenerator(SDL_Renderer* renderer,
                           SDL_Texture* groundTex,
                           SDL_Texture* ugTex,
                           std::vector<ground_Class> &prevGround,
                           std::vector<trapSpike>& trap);
// std::vector<ground_Class> cameraMovmentObj(std::vector<ground_Class> ground,float
cameraOffset);

// std::vector<ground_Class> freeMemory(std::vector<ground_Class> ground);
void freeMemory(std::vector<ground_Class>& ground);
void cameraMovmentObj(std::vector<ground_Class>& ground, float
cameraOffset,std::vector<trapSpike>&trap);

void trapTriangle(SDL_Renderer* renderer, std::vector<trapSpike>& trap,
std::vector<ground_Class>& ground, int i, int maxTraps = 4);
void trapFreeMemory(std::vector<trapSpike>&trap);

bool menuScreen(SDL_Renderer* renderer, TTF_Font* font);
LoginScreenAction runLoginScreen(SDL_Renderer* renderer, TTF_Font* font);
RegisterScreenAction runRegisterScreen(SDL_Renderer* renderer, TTF_Font* font);
void runScoreMenu(SDL_Renderer* renderer, TTF_Font* font);

void physicsDevelop(player& player1,
                    std::vector<ground_Class>& g1,
                    float deltaTime,
                    const Uint8* keyboardState);
// ... excerpt trimmed for the report ...
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 15 - RANDOM NUMBER GENERATION

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is ground.cpp, traps.cpp. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

RNG Flowchart

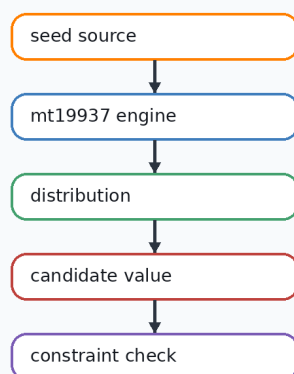


Figure 15.1 RNG

flowchart

The engine and distribution split seed generation from number range logic.

Probability Systems

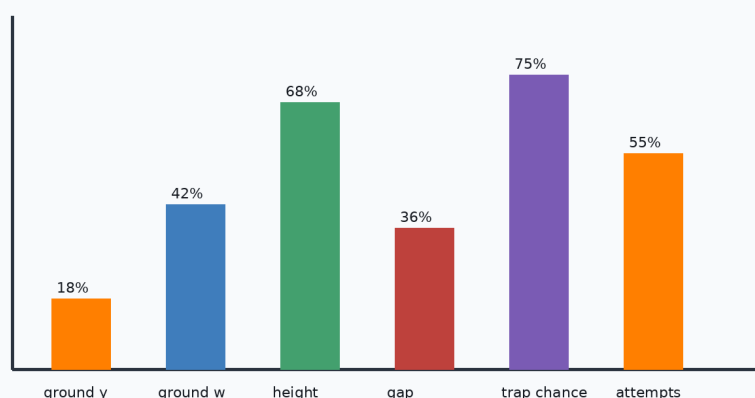


Figure 15.2 Probability

Different random variables shape difficulty and platform rhythm.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAII-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

Randomness controls ground y-position, width, height, gap, and trap placement. The current implementation uses `std::random_device` and `std::mt19937` for modern C++ random generation, while the chapter also explains beginner `srand/rand` concepts for comparison.

Code Excerpt: `ground.cpp`

```
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>

#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include<random>
#include "functions.h"

namespace
{
    int clampInt(int value, int minValue, int maxValue)
    {
        if (value < minValue)
        {
            return minValue;
        }
    }
}
```

```

        if (value > maxValue)
        {
            return maxValue;
        }
        return value;
    }
}

```

```

std::vector<ground_Class> ground_generator(SDL_Renderer* renderer,SDL_Texture*
groundTex,SDL_Texture* ugTex)
{
    int screenW = currentScreenWidth;
    int screenH = currentScreenHeight;
    getScreenSize(renderer, screenW, screenH);

    std::random_device rd;
    std::mt19937 gen(rd());

    const int minY = clampInt((int)(screenH * 0.40f), 160, screenH - 260);
    const int maxY = clampInt((int)(screenH * 0.72f), minY + 50, screenH - 120);
    const int minW = clampInt((int)(screenW * 0.52f), 700, 1700);
    const int maxW = clampInt((int)(screenW * 0.95f), minW, 3000);
    const int maxH = clampInt((int)(screenH * 0.30f), 140, 460);

    std::uniform_int_distribution<> disty(minY, maxY);
    std::uniform_int_distribution<> distw(minW, maxW);
    std::uniform_int_distribution<> disth(50, maxH);

    int x = 0;
    int y = disty(gen);
    int w = distw(gen);
    int h = disth(gen);

    std::vector<ground_Class> ground;

    ground.emplace_back(
        x, y, w, h,
        groundTex,ugTex
    );
    ground[0].first_ground=true;

    return ground;
}

```

```

void groundRandomgenerator(SDL_Renderer* renderer,SDL_Texture* groundTex,SDL_Texture*

```

```
ugTex, std::vector<ground_Class> &prevGround, std::vector<trapSpike>& trap)
{
    if (prevGround.empty())
// ... excerpt trimmed for the report ...
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: traps.cpp

```
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>
#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include<random>
#include "functions.h"

void trapTriangle(SDL_Renderer* renderer, std::vector<trapSpike>& trap,
std::vector<ground_Class>& ground, int i, int maxTraps)
{
    if (ground[i].first_ground || ground[i].trapCount >= maxTraps)
    {
        return;
    }

    int screenW = currentScreenWidth;
    int screenH = currentScreenHeight;
    getScreenSize(renderer, screenW, screenH);
    (void)screenH;

    std::random_device rd;
    std::mt19937 gen(rd());

    // Add a safety limit to prevent infinite looping if conditions are hard to meet
    int attempts = 0;
```

```

    const int maxAttempts = 100;

    while (attempts < maxAttempts)
    {
        attempts++;
        std::uniform_int_distribution<> chances(0, 1000);

        // Probability check
        if (chances(gen) < 100) // Adjusted probability for more consistent generation
        {
            const int trapSize = scaledTrapSize();
            const int margin = std::max(100, trapSize * 2);
            if (ground[i].w <= margin * 2)
            {
                return;
            }

            std::uniform_int_distribution<> distp(ground[i].x + margin,
                                                    (ground[i].x + ground[i].w) - margin);
            int spikeX = distp(gen);

            // Check for overlap with existing traps
            bool overlapping = false;
            const int minTrapSpacing = std::min(300, std::max(trapSize * 2, screenW / 10));
            for (const auto& existingTrap : trap)
            {
                if (spikeX - existingTrap.x <= minTrapSpacing &&
                    spikeX - existingTrap.x >= -minTrapSpacing)
                {
                    overlapping = true;
                    break;
                }
            }

            if (!overlapping && ground[i].trapCount < maxTraps)
            {
                ground[i].trapCount++;
                trap.emplace_back(spikeX, ground[i].y, trapSize, trapSize);
                return; // Successfully added one trap
            }
        }
    }
    // ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 16 - LIBRARIES USED

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is main.cpp, functions.h. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

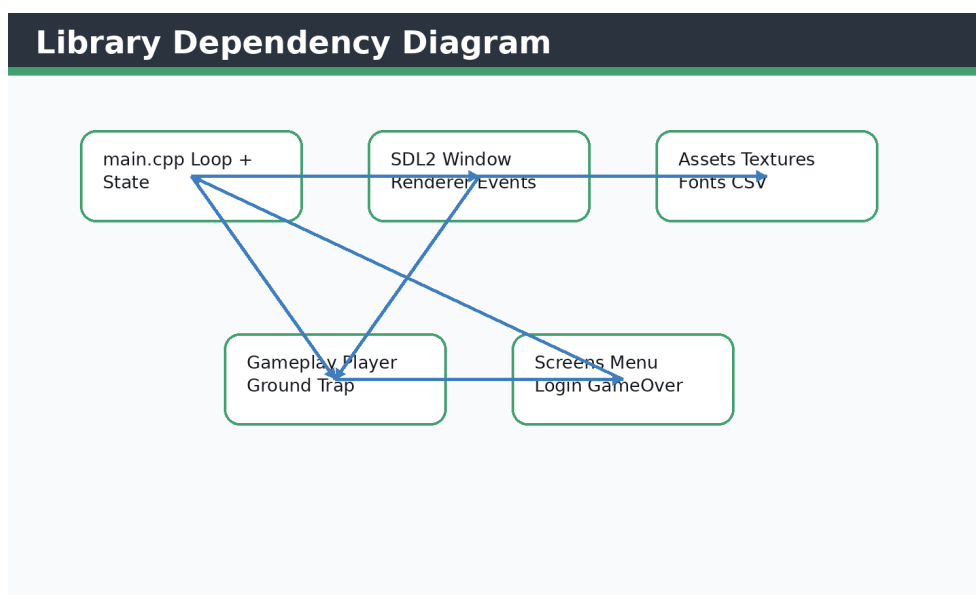


Figure 16.1 Library

dependency diagram

Project modules depend on SDL extensions and standard C++ libraries.

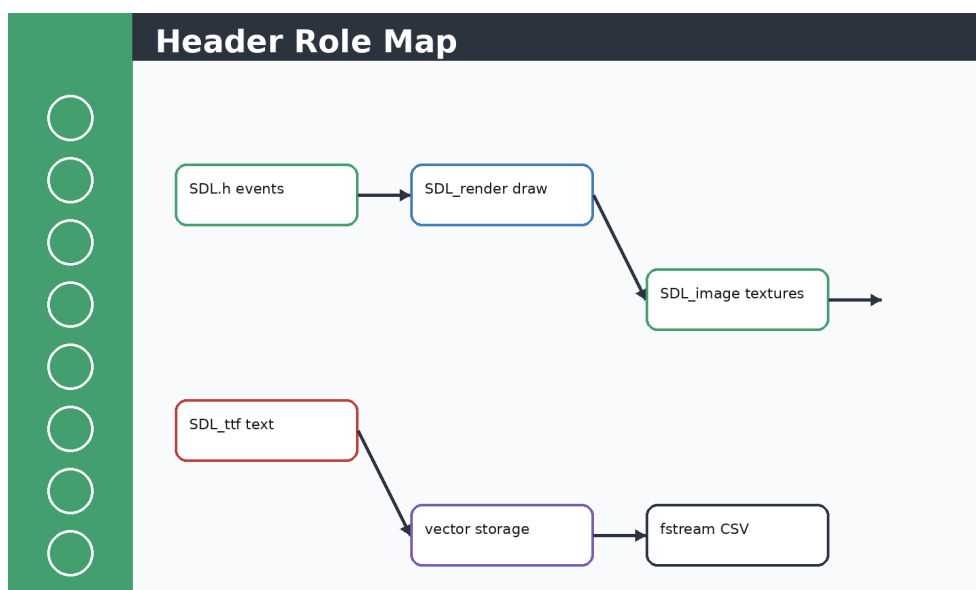


Figure 16.2 Header

role map

Headers map directly to rendering, input, text, storage, and containers.

SDL2 and extension functions used by Jumpa Verse

<code>SDL_Init(Uint32 flags)</code>	int	Initializes SDL subsystems; called in <code>init()</code> before creating the window.
<code>SDL_CreateWindow(title,x,y,w,h,flags)</code>	SDL_Window*	Allocates the native game window and applies fullscreen/resizable flags.
<code>SDL_CreateRenderer(window,index,flags)</code>	SDL_Renderer*	Creates the accelerated rendering context used by all draw calls.
<code>SDL_PollEvent(SDL_Event*)</code>	int	Pulls queued keyboard, mouse, text, quit, and window events inside each loop.
<code>SDL_GetKeyboardState(int*)</code>	const Uint8*	Returns the current keyboard state array used by <code>physicsDevelop()</code> .
<code>SDL_GetMouseState(int*,int*)</code>	Uint32	Reads mouse position/buttons for menu hover and player jump shortcuts.
<code>SDL_SetRenderDrawColor(renderer,r,g,b,a)</code>	int	Selects the drawing color for clear, rectangles, borders, and UI.
<code>SDL_RenderClear(renderer)</code>	int	Clears the current back buffer before drawing the next frame.
<code>SDL_RenderFillRect(renderer, rect)</code>	int	Draws filled UI buttons, player body, and panels.
<code>SDL_RenderDrawRect(renderer, rect)</code>	int	Draws borders for score boxes, controls, and hitbox-style UI.
<code>SDL_RenderCopy(renderer, texture, src, dst)</code>	int	Copies texture pixels into a target rectangle for ground and text.

<code>SDL_RenderPresent(renderer)</code>	void	Swaps the rendered back buffer to the visible screen.
<code>IMG_Init(flags)</code>	int	Initializes SDL_image JPEG support for ground texture loading.
<code>IMG_LoadTexture(renderer, path)</code>	SDL_Texture*	Loads image files directly into GPU texture resources.
<code>TTF_OpenFont(path, size)</code>	TTF_Font*	Loads typefaces used by menus, score display, and screens.
<code>TTF_RenderText_Blended(font, text, color)</code>	SDL_Surface*	Rasterizes text into a temporary CPU surface.
<code>SDL_CreateTextureFromSurface(renderer, surface)</code>	SDL_Texture*	Uploads the text surface into a GPU texture.
<code>filledTrigonRGBA(renderer, ...)</code>	int	SDL2_gfx filled triangle used for solid spike traps.
<code>trigonRGBA(renderer, ...)</code>	int	SDL2_gfx outline triangle used for trap edges.
<code>SDL_SetWindowFullscreen(window, flags)</code>	int	Toggles desktop fullscreen and windowed mode in custom controls.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAII-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

The project includes SDL core headers, `SDL_render`, `SDL_video`, `SDL_image`, `SDL_ttf`, `SDL2_gfx`,

iostream, iterator, vector, string/string.h, algorithm, stdlib, fstream, sstream, and project headers. Each library contributes a narrow role.

Code Excerpt: main.cpp

```
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>
#include<vector>
#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include "functions.h"
#include<stdlib.h>
#include<SDL2/SDL_ttf.h>
float cammeraDash=0;
bool exitb = false;
int main()
{
    SDL_Window* window=nullptr;
    SDL_Renderer* renderer=nullptr;

    player player1;

    if(!init(window,renderer))
    {
        std::cerr<<"FAILED TO START "<<std::endl;
        return -1;
    }

    float cammera_offSet=0.5f; //no 2d camera in sdl2 so we create an illusion
    // ground_Class
    ground(100,900,500,100,renderer,"Images_textures/ground.jpeg","Images_textures/
underGround.jpeg");

    // ground=groundRandomgenerator(ground,renderer);
    SDL_Texture* groundTex = IMG_LoadTexture(renderer, "Images_textures/ground.jpeg");
    SDL_Texture* ugTex = IMG_LoadTexture(renderer, "Images_textures/underGround.jpeg");
    TTF_Font *font =TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf",100);
    std::vector<trapSpike> trap;
```

```

    std::vector<ground_Class> ground;
    Score score;
    GameOver gameOver;
    auto resetGameplay = [&]()
    {
        player1 = player();
        cammera_offSet = 0.5f;
        trap.clear();
        ground = ground_generator(renderer, groundTex, ugTex);
        groundRandomgenerator(renderer, groundTex, ugTex, ground, trap);
        // ground.push_back(temp);
        resetScore(score);
        gameOver.reset();
    };

    initializeScore(score);
    resetGameplay();
    bool quit=false;
    SDL_Event eventManager;
    Uint32 lastTime = SDL_GetTicks();
    int i=0;
    int count=0;
    const float targetFrameTime = 1000.0f / 60.0f; // 16.67 ms
    bool menu=false;
    while(!quit)
    {

        Uint32 frameStart = SDL_GetTicks();
        int screenW = currentScreenWidth;
        int screenH = currentScreenHeight;
        getScreenSize(renderer, screenW, screenH);
        // ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: functions.h

```

#pragma once
#include <SDL2/SDL.h>
#include <vector>
#include <SDL2/SDL_ttf.h>

```

```
#include <string>
#include <cctype>

inline std::string toUpperUsername(const std::string& username)
{
    std::string upperUsername = username;
    for (char& character : upperUsername)
    {
        character = (char)std::toupper((unsigned char)character);
    }
    return upperUsername;
}

extern bool exitb;
extern float cammeraDash;
extern std::string currentPlayerUsername;

constexpr int GAME_WIDTH = 1980;
constexpr int GAME_HEIGHT = 1260;
extern int currentScreenWidth;
extern int currentScreenHeight;

class ground_Class;
class trapSpike;
class player;
class Score;
class GameOver;
enum class GameOverAction;
enum class DeathCause;
enum class LoginScreenAction;
enum class RegisterScreenAction;
bool init(SDL_Window*& Window, SDL_Renderer*& renderer);
void close(SDL_Window* window, SDL_Renderer* renderer);
void getScreenSize(SDL_Renderer* renderer, int& screenW, int& screenH);
bool handleWindowControlEvent(SDL_Renderer* renderer, const SDL_Event& event);
void renderWindowControls(SDL_Renderer* renderer);

std::vector<ground_Class> ground_generator(SDL_Renderer* renderer, SDL_Texture*
groundTex, SDL_Texture* ugTex);

// ground_Class groundRandomgenerator(SDL_Renderer* renderer, SDL_Texture* groundTex, SDL_Texture*
ugTex, std::vector<ground_Class> &prevGround, std::vector<trapSpike>& trap);

void groundRandomgenerator(SDL_Renderer* renderer,
```

```

        SDL_Texture* groundTex,
        SDL_Texture* ugTex,
        std::vector<ground_Class> &prevGround,
        std::vector<trapSpike>& trap);
// std::vector<ground_Class> cameraMovmentObj(std::vector<ground_Class> ground, float
cameraOffset);

// std::vector<ground_Class> freeMemory(std::vector<ground_Class> ground);
void freeMemory(std::vector<ground_Class>& ground);
void cameraMovmentObj(std::vector<ground_Class>& ground, float
cameraOffset, std::vector<trapSpike>& trap);

void trapTriangle(SDL_Renderer* renderer, std::vector<trapSpike>& trap,
std::vector<ground_Class>& ground, int i, int maxTraps = 4);
void trapFreeMemory(std::vector<trapSpike>& trap);

bool menuScreen(SDL_Renderer* renderer, TTF_Font* font);
LoginScreenAction runLoginScreen(SDL_Renderer* renderer, TTF_Font* font);
RegisterScreenAction runRegisterScreen(SDL_Renderer* renderer, TTF_Font* font);
void runScoreMenu(SDL_Renderer* renderer, TTF_Font* font);

void physicsDevelop(player& player1,
        std::vector<ground_Class>& g1,
        float deltaTime,
        const Uint8* keyboardState);
// ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 17 - FULLSCREEN IMPLEMENTATION

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is `Error_Handlers.cpp`, `functions.h`. The code favors simple classes, vectors, `SDL_Rect` geometry, shared `SDL_Texture` pointers, and explicit event polling.

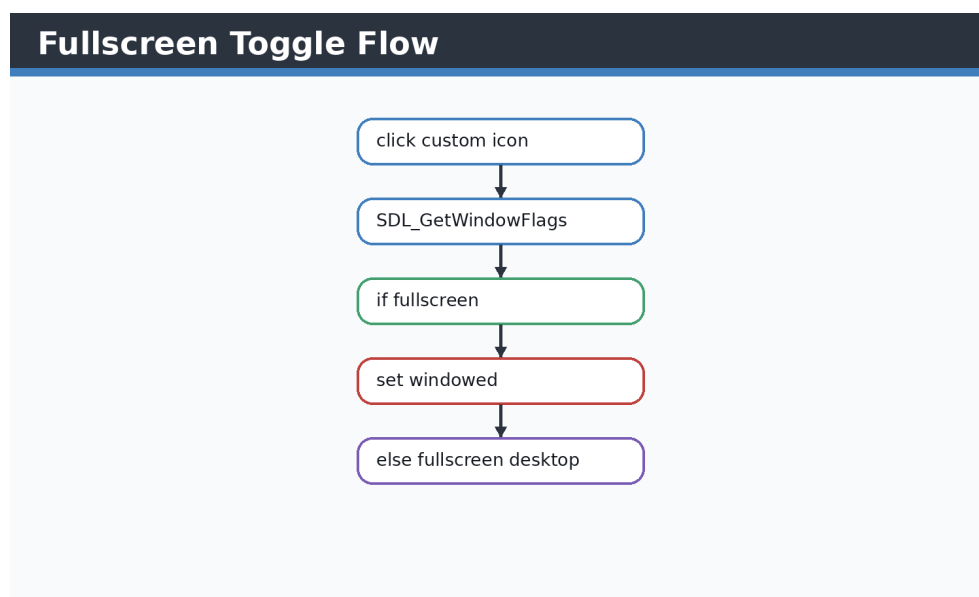


Figure 17.1 Fullscreen

diagram

The toggle checks current flags before changing the window mode.

Resolution Comparison

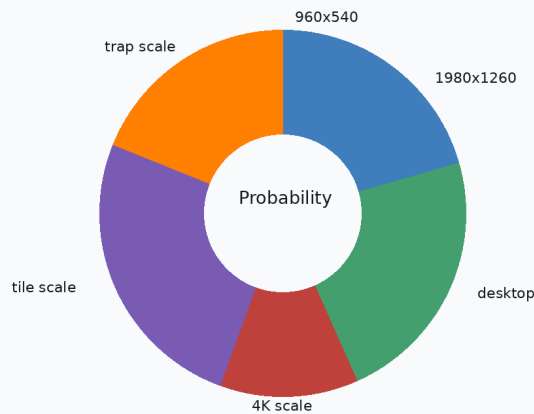


Figure 17.2 Resolution

comparison

Screen-dependent scaling functions keep gameplay readable across display sizes.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAII-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

Fullscreen mode is enabled at window creation with `SDL_WINDOW_FULLSCREEN_DESKTOP` and toggled later through custom window controls using `SDL_SetWindowFullscreen`. Windowed mode restores the configured `GAME_WIDTH` and `GAME_HEIGHT`.

Code Excerpt: `Error_Handlers.cpp`

```
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include<vector>
#include<string.h>
#include<vector>
#include<algorithm> //For basic algorithm like sorting
```

```
#include "object.h"
#include "functions.h"
#include <SDL2/SDL_image.h>

bool init(SDL_Window*& Window, SDL_Renderer *&renderer); //Check
void close(SDL_Window* window, SDL_Renderer* renderer);

int currentScreenWidth = GAME_WIDTH;
int currentScreenHeight = GAME_HEIGHT;

bool init(SDL_Window*& Window, SDL_Renderer *&renderer)
{
    if(SDL_Init(SDL_INIT_VIDEO)<0)
    {
        std::cerr<<"SDL couldnt be initialized! \t\t\t Error Type:\t"<<SDL_GetError()<<std::endl;
        return false;
    }
    if(TTF_Init()<0)
    {
        std::cout<<"TTF NOT FOUND"<<std::endl;
        return false;
    }

    if (!(IMG_Init(IMG_INIT_JPG) & IMG_INIT_JPG))
    {
        std::cerr << "SDL_image could not initialize! Error: " << IMG_GetError() << std::endl;
        return false;
    }

    SDL_DisplayMode displayMode;
    if (SDL_GetCurrentDisplayMode(0, &displayMode) == 0)
    {
        currentScreenWidth = displayMode.w;
        currentScreenHeight = displayMode.h;
    }

    Window = SDL_CreateWindow("Jumpa Verse",
                               SDL_WINDOWPOS_CENTERED,
                               SDL_WINDOWPOS_CENTERED,
                               currentScreenWidth,
                               currentScreenHeight,
                               SDL_WINDOW_SHOWN | SDL_WINDOW_RESIZABLE |
```

```

SDL_WINDOW_FULLSCREEN_DESKTOP);

    if(!Window)
    {
        std::cerr<<"SDL WINDOW COULDNT START \t\t\tERROR TYPE:\t"<<SDL_GetError()<<std::endl;
        return false;
    }

    renderer=SDL_CreateRenderer(Window,-1,SDL_RENDERER_ACCELERATED);

    if(!renderer)
    {
        std::cerr<<"RENDERER COULDNT LOAD!\t\t\tSDL ERROR:\t"<<SDL_GetError()<<std::endl;
        return false;
    }

    SDL_SetWindowMinimumSize(Window, 960, 540);

    return true;
// ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: functions.h

```

#pragma once
#include <SDL2/SDL.h>
#include <vector>
#include <SDL2/SDL_ttf.h>
#include <string>
#include <cctype>

inline std::string toUpperUsername(const std::string& username)
{
    std::string upperUsername = username;
    for (char& character : upperUsername)
    {
        character = (char)std::toupper((unsigned char)character);
    }
    return upperUsername;
}

```



```
extern bool exitb;
extern float cammeraDash;
extern std::string currentPlayerUsername;

constexpr int GAME_WIDTH = 1980;
constexpr int GAME_HEIGHT = 1260;
extern int currentScreenWidth;
extern int currentScreenHeight;

class ground_Class;
class trapSpike;
class player;
class Score;
class GameOver;
enum class GameOverAction;
enum class DeathCause;
enum class LoginScreenAction;
enum class RegisterScreenAction;
bool init(SDL_Window*& Window, SDL_Renderer*& renderer);
void close(SDL_Window* window, SDL_Renderer* renderer);
void getScreenSize(SDL_Renderer* renderer, int& screenW, int& screenH);
bool handleWindowControlEvent(SDL_Renderer* renderer, const SDL_Event& event);
void renderWindowControls(SDL_Renderer* renderer);

std::vector<ground_Class> ground_generator(SDL_Renderer* renderer,SDL_Texture*
groundTex,SDL_Texture* ugTex);

// ground_Class groundRandomgenerator(SDL_Renderer* renderer,SDL_Texture* groundTex,SDL_Texture*
ugTex,std::vector<ground_Class> &prevGround,std::vector<trapSpike>& trap);

void groundRandomgenerator(SDL_Renderer* renderer,
                           SDL_Texture* groundTex,
                           SDL_Texture* ugTex,
                           std::vector<ground_Class> &prevGround,
                           std::vector<trapSpike>& trap);
// std::vector<ground_Class> cameraMovmentObj(std::vector<ground_Class> ground,float
cameraOffset);

// std::vector<ground_Class> freeMemory(std::vector<ground_Class> ground);
void freeMemory(std::vector<ground_Class>& ground);
void cameraMovmentObj(std::vector<ground_Class>& ground, float
cameraOffset,std::vector<trapSpike>&trap);
```

```
void trapTriangle(SDL_Renderer* renderer, std::vector<trapSpike>& trap,
std::vector<ground_Class>& ground, int i, int maxTraps = 4);
void trapFreeMemory(std::vector<trapSpike>&trap);

bool menuScreen(SDL_Renderer* renderer, TTF_Font* font);
LoginScreenAction runLoginScreen(SDL_Renderer* renderer, TTF_Font* font);
RegisterScreenAction runRegisterScreen(SDL_Renderer* renderer, TTF_Font* font);
void runScoreMenu(SDL_Renderer* renderer, TTF_Font* font);

void physicsDevelop(player& player1,
                    std::vector<ground_Class>& g1,
                    float deltaTime,
                    const Uint8* keyboardState);
// ... excerpt trimmed for the report ...
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 18 - CLAMPING SYSTEM

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is ground.cpp, player.cpp, main.cpp. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

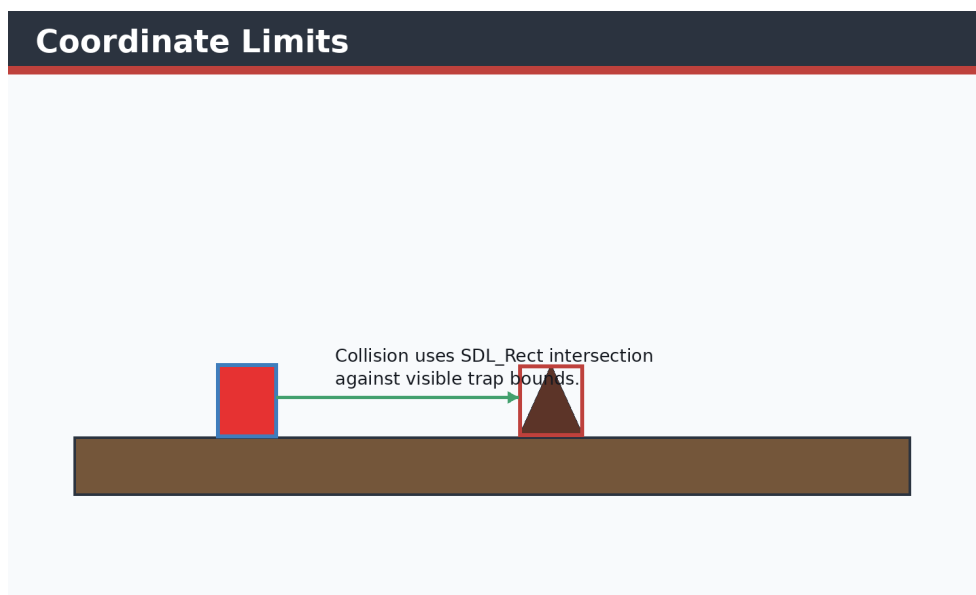


Figure 18.1 Coordinate

limit diagram

Clamping prevents positions and dimensions from leaving useful gameplay boundaries.

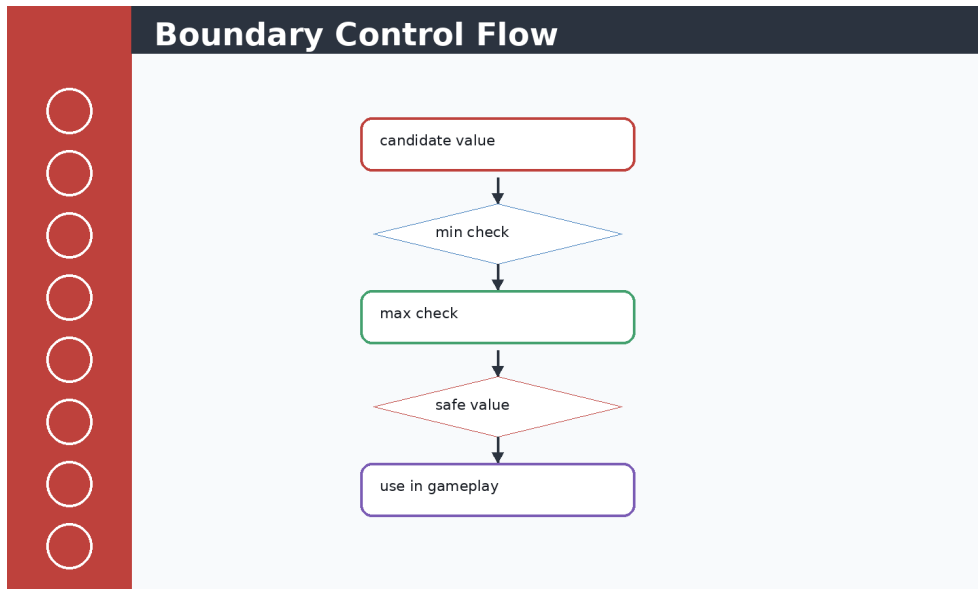


Figure 18.2 Boundary

visual

The clampInt helper in ground.cpp is representative of this safety pattern.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAII-style cleanup where SDL objects are manually destroyed. STL concepts include `std::vector` storage, push/emplace operations, erase-based recycling, `std::string` text state, `fstream` streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

Clamping restricts values to safe ranges. The project clamps generation ranges, player x-position, `deltaTime`, tile sizes, and screen-dependent platform dimensions so random generation and physics remain playable.

Code Excerpt: ground.cpp

```
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>
```

```
#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include<random>
#include "functions.h"

namespace
{
    int clampInt(int value, int minValue, int maxValue)
    {
        if (value < minValue)
        {
            return minValue;
        }
        if (value > maxValue)
        {
            return maxValue;
        }
        return value;
    }
}

std::vector<ground_Class> ground_generator(SDL_Renderer* renderer, SDL_Texture*
groundTex, SDL_Texture* ugTex)
{
    int screenW = currentScreenWidth;
    int screenH = currentScreenHeight;
    getScreenSize(renderer, screenW, screenH);

    std::random_device rd;
    std::mt19937 gen(rd());

    const int minY = clampInt((int)(screenH * 0.40f), 160, screenH - 260);
    const int maxY = clampInt((int)(screenH * 0.72f), minY + 50, screenH - 120);
    const int minW = clampInt((int)(screenW * 0.52f), 700, 1700);
    const int maxW = clampInt((int)(screenW * 0.95f), minW, 3000);
    const int maxH = clampInt((int)(screenH * 0.30f), 140, 460);

    std::uniform_int_distribution<> disty(minY, maxY);
    std::uniform_int_distribution<> distw(minW, maxW);
    std::uniform_int_distribution<> disth(50, maxH);

    int x = 0;
    int y = disty(gen);
```

```

        int w = distw(gen);
        int h = disth(gen);

        std::vector<ground_Class> ground;

        ground.emplace_back(
            x, y, w, h,
            groundTex,ugTex
        );
        ground[0].first_ground=true;

        return ground;
    }

void groundRandomgenerator(SDL_Renderer* renderer,SDL_Texture* groundTex,SDL_Texture*
ugTex,std::vector<ground_Class> &prevGround,std::vector<trapSpike>& trap)
{
    if (prevGround.empty())
// ... excerpt trimmed for the report ...

```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Code Excerpt: player.cpp

```

#include <SDL2/SDL_render.h>
#include <SDL2/SDL_scancode.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include <ostream>
#include<vector>
#include<string.h>
#include<vector>
#include<algorithm>///For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include "functions.h"
#include<stdlib.h>
#include<SDL2/SDL_ttf.h>

```

```

#include<limits>
int i=0;
void physicsDevelop(player& player1,
                    std::vector<ground_Class>& g1,
                    float deltaTime,
                    const Uint8* keyboardState)
{
    Uint32 mouseState = SDL_GetMouseState(NULL, NULL);

    if (deltaTime <= 0.0f)
    {
        return;
    }

    bool jumpPressed = keyboardState[SDL_SCANCODE_SPACE] ||
                        keyboardState[SDL_SCANCODE_W] ||
                        keyboardState[SDL_SCANCODE_UP];

    bool characterDown=keyboardState[SDL_SCANCODE_LCTRL] ||
                        keyboardState[SDL_SCANCODE_S] ||
                        keyboardState[SDL_SCANCODE_DOWN];

    bool characterdash=keyboardState[SDL_SCANCODE_LSHIFT] ||
                        keyboardState[SDL_SCANCODE_D] ||
                        keyboardState[SDL_SCANCODE_RIGHT];
    bool backward=keyboardState[SDL_SCANCODE_A] ||
                  keyboardState[SDL_SCANCODE_LEFT] ||
                  keyboardState[SDL_SCANCODE_LALT];

    bool leftClick  = mouseState & SDL_BUTTON(SDL_BUTTON_LEFT);
    bool rightClick = mouseState & SDL_BUTTON(SDL_BUTTON_RIGHT);
    bool middleClick= mouseState & SDL_BUTTON(SDL_BUTTON_MIDDLE);

    // if (mouseState & SDL_BUTTON(SDL_BUTTON_RIGHT))
    // {
    //     std::cout << "Right click detected\n";
    // }

    if ((jumpPressed || leftClick)&& player1.onGround)
    {
        player1.jumpVelocity = player1.jumpPower;
        player1.onGround = false;
    }

    if (characterDown||rightClick)
    {
        player1.gravity+=2000;
    }
}

```

```
    }  
    const int playerMoveRight = std::min(9, std::max(5, currentScreenWidth / 360));  
    const int playerMoveLeft = std::min(14, std::max(8, currentScreenWidth / 240));  
    const int rightLimit = currentScreenWidth - player1.character.w;  
    if(characterdash && (player1.character.x+player1.character.w)<currentScreenWidth)  
    {  
        player1.character.x += playerMoveRight;  
        if (player1.character.x > rightLimit)  
        {  
            // ... excerpt trimmed for the report ...  
        }  
    }
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Chapter 19 - MAIN.CPP COMPLETE FLOW

Theory: this chapter starts from the beginner concept, then connects it to the concrete C++ and SDL2 implementation used in Jumpa Verse.

Concept: the module is built as a small real-time system where state is stored in C++ objects, updated once per frame, and drawn through SDL2 renderer commands.

Implementation: the relevant source surface for this chapter is main.cpp. The code favors simple classes, vectors, SDL_Rect geometry, shared SDL_Texture pointers, and explicit event polling.

Main.cpp Execution Flow

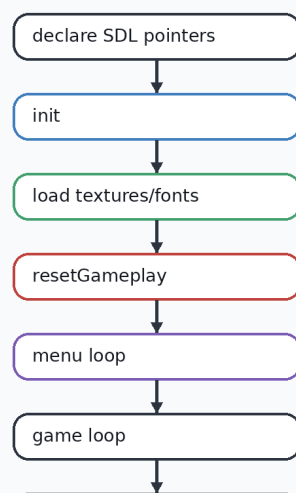


Figure 19.1 Main loop

diagram

The line-by-line structure of main.cpp is a staged real-time application lifecycle.

Physics and Rendering Cycle

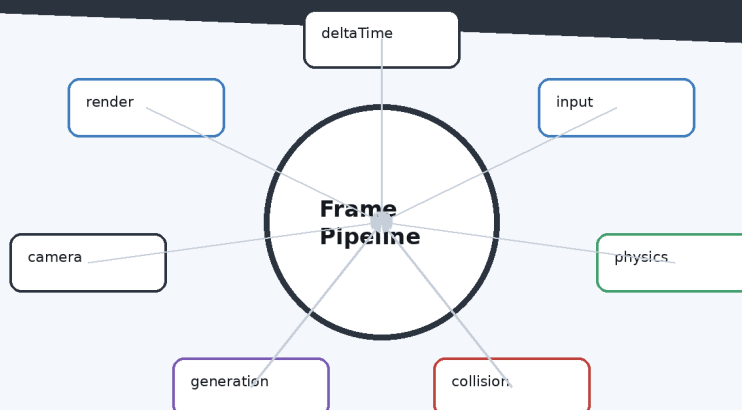


Figure 19.2 Execution

flowchart

Each frame repeats the same update and draw order to keep behavior deterministic enough for gameplay.

Line-by-line summary: SDL pointers are initialized to null; player and gameplay containers are created; init() validates subsystems; textures and fonts are loaded; resetGameplay clears containers and recreates the starting world; the menu loop blocks gameplay until Play/Login succeeds; deltaTime is capped; events are polled; keyboard state drives physicsDevelop; death checks trigger GameOver; score updates only while alive; rendering clears the sky, generates enough future ground, frees old objects, applies camera movement, renders ground/traps/player/score/window controls, presents the frame, delays to target 60 FPS, and finally close() destroys SDL resources.

Concepts Before Code

C++ concepts used here include classes, constructors, references, pointers, enums, namespaces, stack variables, dynamic containers, and RAII-style cleanup where SDL objects are manually destroyed. STL concepts include std::vector storage, push/emplace operations, erase-based recycling, std::string text state, fstream streams, sorting, and iterating by index or reference. SDL2 concepts include renderer ownership, event queues, rectangles as geometry, texture resources, surfaces as temporary CPU bitmaps, and frame presentation.

Implementation Notes

main.cpp owns the startup, gameplay reset lambda, menu gate, per-frame timing, event polling, physics calls, collision/death checks, score update, generation buffer, cleanup, camera movement, rendering, FPS throttling, and shutdown.

Code Excerpt: main.cpp

```
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>
#include<vector>
#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include "functions.h"
#include<stdlib.h>
#include<SDL2/SDL_ttf.h>
```

```

float cammeraDash=0;
bool exitb = false;
int main()
{
    SDL_Window* window=nullptr;
    SDL_Renderer* renderer=nullptr;

    player player1;

    if(!init(window,renderer))
    {
        std::cerr<<"FAILED TO START "<<std::endl;
        return -1;
    }

    float cammera_offSet=0.5f; //no 2d camera in sdl2 so we create an illusion
    // ground_Class
    ground(100,900,500,100,renderer,"Images_textures/ground.jpeg","Images_textures/
underGround.jpeg");

    // ground=groundRandomgenerator(ground,renderer);
    SDL_Texture* groundTex = IMG_LoadTexture(renderer, "Images_textures/ground.jpeg");
    SDL_Texture* ugTex = IMG_LoadTexture(renderer, "Images_textures/underGround.jpeg");
    TTF_Font *font =TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf",100);
    std::vector<trapSpike> trap;
    std::vector<ground_Class> ground;
    Score score;
    GameOver gameOver;
    auto resetGameplay = [&]()
    {
        player1 = player();
        cammera_offSet = 0.5f;
        trap.clear();
        ground = ground_generator(renderer, groundTex, ugTex);
        groundRandomgenerator(renderer, groundTex, ugTex, ground, trap);
        // ground.push_back(temp);
        resetScore(score);
        gameOver.reset();
    };

    initializeScore(score);
    resetGameplay();
    bool quit=false;

```

```
    SDL_Event eventManager;
    Uint32 lastTime = SDL_GetTicks();
    int i=0;
    int count=0;
    const float targetFrameTime = 1000.0f / 60.0f; // 16.67 ms
    bool menu=false;
    while(!quit)
    {

        Uint32 frameStart = SDL_GetTicks();
        int screenW = currentScreenWidth;
        int screenH = currentScreenHeight;
        getScreenSize(renderer, screenW, screenH);
        // ... excerpt trimmed for the report ...
    }
```

Execution flow: the snippet participates in the game loop by reading current state, calculating next state, then issuing render or storage operations. Memory flow: SDL resources are referenced by pointer, while gameplay objects and rectangles are copied or moved inside vectors. Object interaction: main.cpp coordinates these calls so each subsystem remains small and testable by inspection.

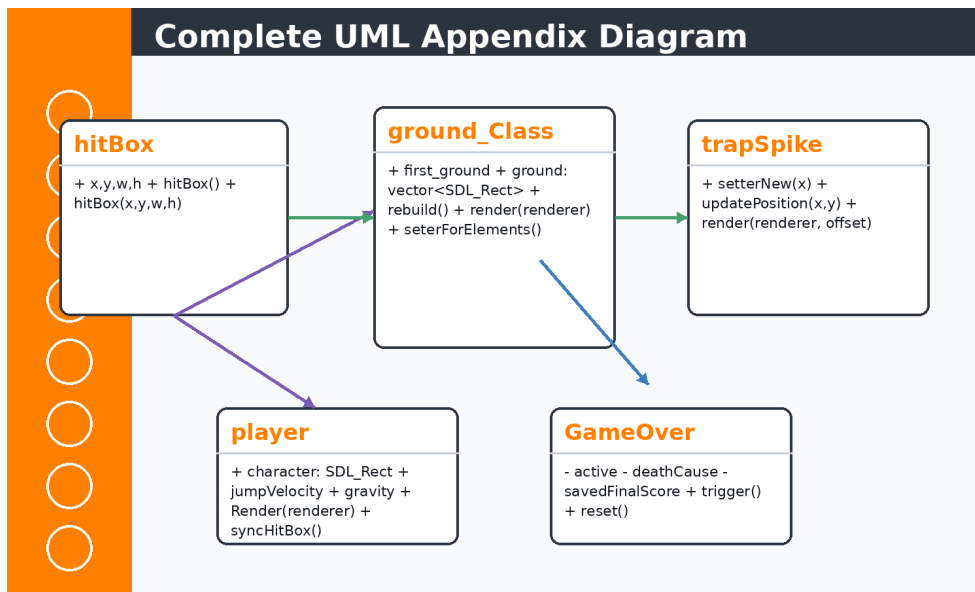
Issues Faced and Solutions

Typical issues in this module include invalid SDL pointers, oversized coordinates, shallow-copy confusion for textures, repeated text texture creation, off-screen objects growing memory, and input state transitions. The implemented solution pattern is to validate pointers, clamp screen-dependent values, share texture pointers without ownership, erase off-screen objects, cap deltaTime, and centralize screen-size queries.

Bibliography

[1] SDL2 Documentation. [2] SDL_image Documentation. [3] SDL_ttf Documentation. [4] SDL2_gfx Primitive Documentation. [5] ISO C++ Standard Library references for vector, string, fstream, algorithm, and random.

Appendix A - UML Appendix



Appendix Figure A.1

Complete UML appendix

This consolidated UML-style diagram summarizes inheritance, association, and dependency relationships used throughout the game.

Appendix B - Code Appendix

main.cpp

```
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>
#include<vector>
#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include "functions.h"
#include<stdlib.h>
#include<SDL2/SDL_ttf.h>
float cammeraDash=0;
bool exitb = false;
int main()
{
    SDL_Window* window=nullptr;
    SDL_Renderer* renderer=nullptr;

    player player1;

    if(!init(window,renderer))
    {
        std::cerr<<"FAILED TO START "<<std::endl;
        return -1;
    }

    float cammera_offSet=0.5f; //no 2d camera in sdl2 so we create an illusion
    // ground_Class
    ground(100,900,500,100,renderer,"Images_textures/ground.jpeg","Images_textures/
underGround.jpeg");

    // ground=groundRandomgenerator(ground,renderer);
    SDL_Texture* groundTex = IMG_LoadTexture(renderer, "Images_textures/ground.jpeg");
```

```

    SDL_Texture* ugTex = IMG_LoadTexture(renderer, "Images_textures/underGround.jpeg");
    TTF_Font *font =TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf",100);
    std::vector<trapSpike> trap;
    std::vector<ground_Class> ground;
    Score score;
    GameOver gameOver;
    auto resetGameplay = [&]()
    {
        player1 = player();
        cammera_offSet = 0.5f;
        trap.clear();
        ground = ground_generator(renderer, groundTex, ugTex);
        groundRandomgenerator(renderer, groundTex, ugTex, ground, trap);
        // ground.push_back(temp);
        resetScore(score);
        gameOver.reset();
    };

    initializeScore(score);
    resetGameplay();
    bool quit=false;
    SDL_Event eventManager;
    Uint32 lastTime = SDL_GetTicks();
    int i=0;
    int count=0;
    const float targetFrameTime = 1000.0f / 60.0f; // 16.67 ms
    bool menu=false;
    while(!quit)
    {

        Uint32 frameStart = SDL_GetTicks();
        int screenW = currentScreenWidth;
        int screenH = currentScreenHeight;
        getScreenSize(renderer, screenW, screenH);

        Uint32 currentTime = SDL_GetTicks();
        float deltaTime = (currentTime - lastTime) / 1000.0f;
        lastTime = currentTime;
        float speed = 2.0f; // pixels per second

        while(!menu && !quit)
        {
            menu=menuScreen(renderer,font);
            if(exitb==true)
            {

```



```
        quit=true;
        continue;
    }
    resetGameplay();
    lastTime = SDL_GetTicks();
}

if (deltaTime > 0.05f)
    deltaTime = 0.05f;

// if (cammera_offSet < -10.0f)
//     cammera_offSet = -10.0f;

if (cammera_offSet > -9.0f)
{
    cammera_offSet -= speed * deltaTime;
}

i++;

while(SDL_PollEvent(&eventManager)!=0)
{
    if (handleWindowControlEvent(renderer, eventManager))
    {
        continue;
    }

    if(eventManager.type==SDL_QUIT)
    {
        quit=true;
    }
}

const Uint8* keyboardState = SDL_GetKeyboardState(NULL);
player1.fallOffset=cammera_offSet;
physicsDevelop(player1, ground, deltaTime, keyboardState);
if (playerFellOff(player1))
{
    gameOver.trigger(DeathCause::FellOff, score.value());
}
else if (playerHitTrap(player1, trap))
{
    gameOver.trigger(DeathCause::HitTrap, score.value());
}
```

```
    }

    if (gameOver.isActive())
    {
        saveFinalScore(gameOver);
        saveLoggedInPlayerScore(gameOver);
        GameOverAction action = runGameOverScreen(renderer, font, gameOver);
        if (exitb)
        {
            quit = true;
            continue;
        }

        resetGameplay();
        lastTime = SDL_GetTicks();

        if (action == GameOverAction::TryAgain)
        {
            menu = true;
        }
        else if (action == GameOverAction::ReturnToMenu)
        {
            menu = false;
        }

        continue;
    }

    bool playerAlive = !gameOver.isActive();
    updateScore(score, deltaTime, playerAlive);

    SDL_SetRenderDrawColor(renderer, 135, 206, 235, 255);
    SDL_RenderClear(renderer);

    const int generationBuffer = std::max(screenW, 1400);
    const int maxGeneratedPlatforms = 10;
    while(!ground.empty() &&
        ground.back().x + ground.back().w < screenW + generationBuffer &&
        (int)ground.size() < maxGeneratedPlatforms)
    {
        groundRandomgenerator(renderer, groundTex, ugTex, ground, trap);
    }
    freeMemory(ground);
```

```

        trapFreeMemory(trap);
    }

    cameraMovmentObj(ground, cammera_offSet+cammeraDash, trap);

    for(int j=0;j<ground.size();j++)
    {
        ground[j].render(renderer);
    }

    for(int j=0;j<trap.size();j++)
    {
        trap[j].render(renderer);
    }

    player1.Render(renderer);
    renderScore(score, renderer, font);
    renderWindowControls(renderer);

    SDL_RenderPresent(renderer);

    static int frames = 0;
    static Uint32 fpsTimer = SDL_GetTicks();

    frames++;

    Uint32 frameTime = SDL_GetTicks() - frameStart;

    if (frameTime < targetFrameTime)
    {
        SDL_Delay((Uint32)(targetFrameTime - frameTime));
    }

    if (SDL_GetTicks() - fpsTimer >= 1000)
    {
        std::cout << "\rFPS: " << frames
        << "   Offset: " << cammera_offSet<< " "<<cammeraDash
        << std::flush;
        frames = 0;
        fpsTimer = SDL_GetTicks();
    }
}

```

```
    }  
    close(window,renderer);  
    return 0;  
}  
}
```

object.h

```
#pragma once  
  
#include <SDL2/SDL.h>  
#include <SDL2/SDL_rect.h>  
#include <SDL2/SDL_image.h>  
#include <SDL2/SDL_render.h>  
#include <SDL2/SDL_surface.h>  
#include <iostream>  
#include <vector>  
#include <string>  
#include <SDL2/SDL2_gfxPrimitives.h> //to import trigons  
#include <SDL2/SDL_ttf.h>  
  
extern int currentScreenWidth;  
extern int currentScreenHeight;  
  
inline int scaledGroundTileSize()  
{  
    if (currentScreenWidth >= 3840)  
    {  
        return 96;  
    }  
    if (currentScreenWidth >= 3200)  
    {  
        return 80;  
    }  
    if (currentScreenWidth >= 2400)  
    {  
        return 64;  
    }  
    return 50;  
}  
  
inline int scaledTrapSize()  
{
```

```
    if (currentScreenWidth >= 3200)
    {
        return 88;
    }
    if (currentScreenWidth >= 2400)
    {
        return 68;
    }
    return 50;
}

inline int scaledPlayerSize()
{
    if (currentScreenWidth >= 3200)
    {
        return 88;
    }
    if (currentScreenWidth >= 2400)
    {
        return 68;
    }
    return 50;
}

class hitBox
{
public:
    int x, y, w, h;

    hitBox(int x, int y, int w, int h)
    {
        this->x = x;
        this->y = y;
        this->w = w;
        this->h = h;
    }
    hitBox();
    // inline optionBoxes::optionBoxes(int x, int y, int w, int h);
};

class ground_Class : public hitBox
{
public:
    bool first_ground=false;
    /*Characterizing ground into 3 types small medium and big*/
```

```
    bool small=false;
    bool big=false;
    bool medium=false;
    int divw = 0;
    int trapCount=0;

    bool trapFill=false;
    std::vector<SDL_Rect> ground;
    std::vector<std::vector<SDL_Rect>> groundUnder;

    SDL_Texture* groundTexture = nullptr; // shared (NOT owned)
    SDL_Texture* ugTexture = nullptr;    // shared (NOT owned)

    int perboxwidth = scaledGroundTileSize();
    int perboxheight = scaledGroundTileSize();

public:
    // Default constructor
    ground_Class() : hitBox(0, 0, 0, 0) {}

    // Main constructor (USES SHARED TEXTURES)
    ground_Class(int ax, int ay, int aw, int ah,
                 SDL_Texture* groundTex,
                 SDL_Texture* ugTex)
    {
        : hitBox(ax, ay, aw, ah),
        groundTexture(groundTex),
        ugTexture(ugTex)
    {
        rebuild();
    }

    // Rebuild geometry only (NO SDL loading here)
    void rebuild()
    {
        divw = w / perboxwidth;
        int rows = h / perboxheight;

        ground.clear();
        groundUnder.clear();

        ground.resize(divw);
        groundUnder.resize(divw);

        for (int i = 0; i < divw; i++)
```

```
    {
        groundUnder[i].resize(rows);
    }

    for (int i = 0; i < divw; i++)
    {
        ground[i] = { x + i * perboxwidth, y, perboxwidth, perboxheight };
        for (int j = 0; j < rows; j++)
        {
            groundUnder[i][j] =
            {
                x + i * perboxwidth,
                y + (j + 1) * perboxheight,
                perboxwidth,
                perboxheight
            };
        }
    }
}

void seterForElements(int newX, int newY, int newW, int newH)
{
    x = newX;
    y = newY;
    w = newW;
    h = newH;

    rebuild();
}

void render(SDL_Renderer* renderer)
{
    if (!groundTexture || !ugTexture)
    {
        std::cerr << "Cannot render: texture is null!\n";
        return;
    }

    for (int i = 0; i < divw; i++)
    {
        SDL_RenderCopy(renderer, groundTexture, nullptr, &ground[i]);

        for (int j = 0; j < (int)groundUnder[i].size(); j++)
        {
            SDL_RenderCopy(renderer, ugTexture, nullptr, &groundUnder[i][j]);
        }
    }
}
```

```
    }  
}  
}
```

```
  
  
};
```

```
class trapSpike : public hitBox  
{  
public:  
    trapSpike() : hitBox(0, 0, 0, 0) {}  
  
    trapSpike(int x, int y, int w, int h) : hitBox(x, y, w, h) {}  
  
    void setterNew(int x)  
    {  
        this->x = x;  
    }  
  
    void render(SDL_Renderer* renderer, float cameraOffset = 0)  
    {  
        int x1 = x + (int)cameraOffset;  
        int y1 = y;  
  
        // Visibility and Overflow Check:  
        // SDL2_gfx uses int16_t (-32768 to 32767).  
        // We only render if it's reasonably near the screen to avoid overflow "streaks".  
        if (x1 < -500 || x1 > currentScreenWidth + 1000)  
        {  
            return;  
        }  
  
        const int trapSize = w > 0 ? w : scaledTrapSize();  
        int x2 = x1 + trapSize;  
        int y2 = y1;  
        int x3 = x1 + trapSize / 2;  
        int y3 = y1 - trapSize;  
  
        filledTrigonRGBA(renderer,  
                           x1, y1,  
                           x2, y2,  
                           x3, y3,  
                           92, 52, 40, 255);
```



```

        trigonRGBA(renderer,
                    x1, y1,
                    x2, y2,
                    x3, y3,
                    40, 22, 18, 255);
    }

    void updatePosition(int x, int y)
    {
        this->x = x;
        this->y = y;
    }
};

class Score
{
public:
    Score();
    ~Score();

    Score(const Score&) = delete;
    Score& operator=(const Score&) = delete;

    void initialize();
    void update(float deltaTime, bool playerAlive);
    void render(SDL_Renderer* renderer, TTF_Font* font);
    void reset();
    int value() const;

private:
    int score = 0;
    float timeAccumulator = 0.0f;
};

enum class DeathCause
{
    None,
    FellOff,
    HitTrap
};

enum class GameOverAction
{
    None,

```

```
// ... excerpt trimmed for the report ...
```

ground.cpp

```
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>

#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include<random>
#include "functions.h"

namespace
{
    int clampInt(int value, int minValue, int maxValue)
    {
        if (value < minValue)
        {
            return minValue;
        }
        if (value > maxValue)
        {
            return maxValue;
        }
        return value;
    }
}

std::vector<ground_Class> ground_generator(SDL_Renderer* renderer, SDL_Texture*
groundTex, SDL_Texture* ugTex)
{
    int screenW = currentScreenWidth;
    int screenH = currentScreenHeight;
    getScreenSize(renderer, screenW, screenH);

    std::random_device rd;
```

```

        std::mt19937 gen(rd());

        const int minY = clampInt((int)(screenH * 0.40f), 160, screenH - 260);
        const int maxY = clampInt((int)(screenH * 0.72f), minY + 50, screenH - 120);
        const int minW = clampInt((int)(screenW * 0.52f), 700, 1700);
        const int maxW = clampInt((int)(screenW * 0.95f), minW, 3000);
        const int maxH = clampInt((int)(screenH * 0.30f), 140, 460);

        std::uniform_int_distribution<> disty(minY, maxY);
        std::uniform_int_distribution<> distw(minW, maxW);
        std::uniform_int_distribution<> disth(50, maxH);

        int x = 0;
        int y = disty(gen);
        int w = distw(gen);
        int h = disth(gen);

        std::vector<ground_Class> ground;

        ground.emplace_back(
            x, y, w, h,
            groundTex,ugTex
        );
        ground[0].first_ground=true;

        return ground;
    }

void groundRandomgenerator(SDL_Renderer* renderer,SDL_Texture* groundTex,SDL_Texture*
ugTex,std::vector<ground_Class> &prevGround,std::vector<trapSpike>& trap)
{
    if (prevGround.empty())
    {
        return;
    }

    int screenW = currentScreenWidth;
    int screenH = currentScreenHeight;
    getScreenSize(renderer, screenW, screenH);

    bool check=false;
    int x,y,w,h;
    std::random_device rd;
    std::mt19937 gen(rd);

```

```

    const int minY = clampInt((int)(screenH * 0.16f), 80, screenH - 260);
    const int maxY = clampInt((int)(screenH * 0.74f), minY + 50, screenH - 120);
    const int minGap = clampInt((int)(screenW * 0.055f), 120, 240);
    const int maxGap = clampInt((int)(screenW * 0.09f), minGap, 380);
    const int minW = clampInt((int)(screenW * 0.30f), 700, 1300);
    const int maxW = clampInt((int)(screenW * 0.86f), minW, 2600);
    const int maxH = clampInt((int)(screenH * 0.30f), 140, 460);

    std::uniform_int_distribution<> distx(prevGround.back().x + prevGround.back().w + minGap,
                                         prevGround.back().x + prevGround.back().w + maxGap);
    std::uniform_int_distribution<> distw(minW, 2400);
    std::uniform_int_distribution<> disth(50, maxH);

    while(!check)
    {
        const int nextMinY = clampInt(prevGround.back().y - (int)(screenH * 0.08f), minY, maxY);
        const int nextMaxY = clampInt(prevGround.back().y + (int)(screenH * 0.48f), nextMinY,
maxY);

        std::uniform_int_distribution<> disty(nextMinY, nextMaxY);
        x = distx(gen);
        y = disty(gen);
        w = distw(gen);
        h = disth(gen);
        if(y <= maxY && y >= minY)
        {
            check=true;
        }
    }
    int settrap = 1;
    prevGround.emplace_back(x, y, w, h, groundTex, ugTex);
    int currentnumberindex = (int)prevGround.size() - 1;

    const int smallPlatformLimit = clampInt((int)(screenW * 0.34f), 750, 1300);
    const int mediumPlatformLimit = clampInt((int)(screenW * 0.66f), smallPlatformLimit + 250,
2100);

    if (prevGround[currentnumberindex].w < smallPlatformLimit)
    {
        prevGround[currentnumberindex].small = true;
        prevGround[currentnumberindex].medium = false;
        prevGround[currentnumberindex].big = false;
        settrap = 1;
    }

```

```

        else if (prevGround[currentnumberindex].w < mediumPlatformLimit)
        {
            prevGround[currentnumberindex].small = false;
            prevGround[currentnumberindex].medium = true;
            prevGround[currentnumberindex].big = false;
            settrap = 3;
        }
        else
        {
            prevGround[currentnumberindex].small = false;
            prevGround[currentnumberindex].medium = false;
            prevGround[currentnumberindex].big = true;
            settrap = 6;
        }

        for (int j = 0; j < settrap; j++)
        {
            trapTriangle(renderer, trap, prevGround, currentnumberindex, settrap);
        }
    }
}

```

```

// void groundRandomgenerator(SDL_Renderer* renderer,
//                             SDL_Texture* groundTex,
//                             SDL_Texture* ugTex,
//                             std::vector<ground_Class> &prevGround,
//                             std::vector<trapSpike>& trap)
// {
//     bool check = false;
//     int x, y, w, h;

//     std::random_device rd;
//     std::mt19937 gen(rd());

//     std::uniform_int_distribution<> distx(
//         prevGround.back().x + prevGround.back().w + 120,
//         prevGround.back().x + prevGround.back().w + 300
//     );

//     std::uniform_int_distribution<> distw(300, 2450);
//     std::uniform_int_distribution<> disth(50, 400);

//     for (int i = 0; i < prevGround.size(); i++)
//     {
//         if (check == false)
//         {
//             x = distx();
//             y = disth();
//             w = distw();
//             h = disth();
//             ground_Class g(x, y, w, h);
//             prevGround.push_back(g);
//             check = true;
//         }
//     }
// }

```

```

//      {
//          trapTriangle(renderer, trap, prevGround, i);
//      }

//      while (!check)
//      {
//          std::uniform_int_distribution<> disty(
//              prevGround.back().y - 100,
//              prevGround.back().y + 600
//          );

//          x = distx(gen);
//          y = disty(gen);
//          w = distw(gen);
//          h = disth(gen);

//          if (y < 800 && y > 100)
//          {
//              check = true;
//          }
//      }

//      // ✅ Construct directly inside the vector
//      prevGround.emplace_back(x, y, w, h, groundTex, ugTex);
//  }

void freeMemory(std::vector<ground_Class>& ground)
{
    for (int i = 0; i < (int)ground.size(); )
    {
        // ✅ Remove platforms that are fully off-screen to the LEFT
        if (ground[i].x + ground[i].w < 0)
            ground.erase(ground.begin() + i);
        else
            i++;
    }
}

void dualPath(std::vector<ground_Class>& ground, SDL_Texture* groundTex, SDL_Texture* ugTex)
{
    std::random_device rd;
    std::mt19937 gen;
    std::uniform_int_distribution<> chances (0,1000);

```

player.cpp

```

        bool characterDown=keyboardState[SDL_SCANCODE_LCTRL] ||
        keyboardState[SDL_SCANCODE_S] ||
        keyboardState[SDL_SCANCODE_DOWN];

        bool characterdash=keyboardState[SDL_SCANCODE_LSHIFT] ||
        keyboardState[SDL_SCANCODE_D] ||
        keyboardState[SDL_SCANCODE_RIGHT];
        bool backward=keyboardState[SDL_SCANCODE_A] ||
        keyboardState[SDL_SCANCODE_LEFT] ||
        keyboardState[SDL_SCANCODE_LALT];

        bool leftClick  = mouseState & SDL_BUTTON(SDL_BUTTON_LEFT);
        bool rightClick = mouseState & SDL_BUTTON(SDL_BUTTON_RIGHT);
        bool middleClick= mouseState & SDL_BUTTON(SDL_BUTTON_MIDDLE);

//        if (mouseState & SDL_BUTTON(SDL_BUTTON_RIGHT))
//        {
//            std::cout << "Right click detected\n";
//        }
        if ((jumpPressed || leftClick)&& player1.onGround)
        {
            player1.jumpVelocity = player1.jumpPower;
            player1.onGround = false;
        }
        if (characterDown||rightClick)
        {
            player1.gravity+=2000;
        }

        const int playerMoveRight = std::min(9, std::max(5, currentScreenWidth / 360));
        const int playerMoveLeft = std::min(14, std::max(8, currentScreenWidth / 240));
        const int rightLimit = currentScreenWidth - player1.character.w;
        if(characterdash && (player1.character.x+player1.character.w)<currentScreenWidth)
        {
            player1.character.x += playerMoveRight;
            if (player1.character.x > rightLimit)
            {
                player1.character.x = rightLimit;
            }
        }

        if (backward && player1.character.x>=0)
        {
            player1.character.x -= playerMoveLeft;
            if (player1.character.x < 0)

```



```
    {
        player1.character.x = 0;
    }
}

if((!characterDown && !rightClick) && player1.gravity != 1800)
{
    player1.gravity = 1800;
}

SDL_Rect previous = player1.character;
int nextGroundIndex = -1;
int nextGroundDistance = std::numeric_limits<int>::max();

player1.jumpVelocity += player1.gravity * deltaTime;
player1.character.y += static_cast<int>(player1.jumpVelocity * deltaTime);
player1.onGround = false;

for (size_t i = 0; i < g1.size(); i++)
{
    SDL_Rect groundRect = {g1[i].x, g1[i].y, g1[i].w, g1[i].h};
    int playerRight = player1.character.x + player1.character.w;
    int distanceToNextGround = groundRect.x - playerRight;

    // The next ground must start after the player's right side.
    // If we compare with player.x, the current ground can be selected
    // when the player is standing near its left side.
    if (distanceToNextGround > 0 && distanceToNextGround < nextGroundDistance)
    {
        nextGroundDistance = distanceToNextGround;
        if(g1.size()>i+1)
            nextGroundIndex = static_cast<int>(i+1);
    }

    bool overlapX =(player1.character.x + player1.character.w > groundRect.x) &&
        (player1.character.x < groundRect.x + groundRect.w);
    bool wasAboveGround = previous.y + previous.h <= groundRect.y;
    bool crossedGround = player1.character.y + player1.character.h >= groundRect.y;

    if (overlapX && wasAboveGround && crossedGround && player1.jumpVelocity >= 0.0f)
    {
        player1.character.y = groundRect.y - player1.character.h;
        player1.jumpVelocity = 0.0f;
        player1.onGround = true;
    }
}
```

```
        break;

    }

}

player1.syncHitBox();
}
```

traps.cpp

```
#include <SDL2/SDL_rect.h>
#include <SDL2/SDL_render.h>
#include <SDL2/SDL_video.h>
#include<iostream>
#include<SDL2/SDL.h>
#include <iterator>
#include<vector>
#include<string.h>
#include<algorithm> //For basic algorithm like sorting
#include"object.h"
#include<SDL2/SDL_image.h>
#include<random>
#include "functions.h"

void trapTriangle(SDL_Renderer* renderer, std::vector<trapSpike>& trap,
std::vector<ground_Class>& ground, int i, int maxTraps)
{
    if (ground[i].first_ground || ground[i].trapCount >= maxTraps)
    {
        return;
    }

    int screenW = currentScreenWidth;
    int screenH = currentScreenHeight;
    getScreenSize(renderer, screenW, screenH);
    (void)screenH;

    std::random_device rd;
    std::mt19937 gen(rd());

    // Add a safety limit to prevent infinite looping if conditions are hard to meet
    int attempts = 0;
    const int maxAttempts = 100;
```

```

        while (attempts < maxAttempts)
        {
            attempts++;

            std::uniform_int_distribution<> chances(0, 1000);

            // Probability check
            if (chances(gen) < 100) // Adjusted probability for more consistent generation
            {
                const int trapSize = scaledTrapSize();
                const int margin = std::max(100, trapSize * 2);
                if (ground[i].w <= margin * 2)
                {
                    return;
                }

                std::uniform_int_distribution<> distp(ground[i].x + margin,
                                                    (ground[i].x + ground[i].w) - margin);
                int spikeX = distp(gen);

                // Check for overlap with existing traps
                bool overlapping = false;
                const int minTrapSpacing = std::min(300, std::max(trapSize * 2, screenW / 10));
                for (const auto& existingTrap : trap)
                {
                    if (spikeX - existingTrap.x <= minTrapSpacing &&
                        spikeX - existingTrap.x >= -minTrapSpacing)
                    {
                        overlapping = true;
                        break;
                    }
                }

                if (!overlapping && ground[i].trapCount < maxTraps)
                {
                    ground[i].trapCount++;
                    trap.emplace_back(spikeX, ground[i].y, trapSize, trapSize);
                    return; // Successfully added one trap
                }
            }
        }
    }

void trapFreeMemory(std::vector<trapSpike>& trap)
{

```

```
    for (int i = 0; i < (int)trap.size(); )
    {
        // Remove traps that are fully off-screen to the left
        const int trapSize = trap[i].w > 0 ? trap[i].w : scaledTrapSize();
        if (trap[i].x + trapSize < -trapSize * 4)
            trap.erase(trap.begin() + i);
        else
            i++;
    }
}
```